

2025 年 12 月 19 日

科目名：生産管理

担当教授：大草 孝介

機械学習モデルにおける変数重要度と SHAP 値の比較検証

--- SKAB データを用いた予測決定要因の考察 ---

中央大学 理工学部 ビジネスデータサイエンス学科

学籍番号 23D7101004B

氏名 萩野谷 颯太

1. はじめに

本レポートでは、機械学習モデルの解釈性に着目し、変数重要度(Feature Importance)と SHAP(Shapley Additive exPlanations)値の比較検証を行う。本レポートでは SKAB(Skoltech Anomaly Benchmark)データセットを用いた異常検知モデルを構築および XGBoost、Random Forest、SVM の 3 つのモデル間での予測寄与変数の違いの分析をデータから考察することを目的とした。

2. SKAB データにおける異常検知モデルの比較

2.1. SKAB データの概要と前処理

以下の表 1 に本レポートで用いた SKAB データ(Valve1)の変数を示す。SKAB データとは Skolkovo 科学研究所が提供する循環水ポンプのセンサーデータである。データは循環水ポンプに取り付けられたセンサーから取得した 8 次元時系列データである。

表 1 変数説明(課題 1)

変数名	説明
Accelerometer1RMS	加速度計 1 の RMS 値。振動の強さを示す。
Accelerometer2RMS	加速度計 2 の RMS 値。別の軸の振動。
Current	モータ等の電流値。負荷状態を表す。
Pressure	循環水の圧力値。
Temperature	エンジンの温度
Thermocouple	循環水の温度
Voltage	電圧値。電力供給の状態を示す。
Volume Flow RateRMS	流量の RMS 値。流体の流れの振動。
anomaly	異常フラグ(0:正常、1:異常)。目的変数。
datetime	日時データ
changepoint	変化点フラグ

ここで変数間の相関を確認するために、相関分析を実施する。各変数間の相関行列をヒートマップで表したものを以下の図 1 に示す。

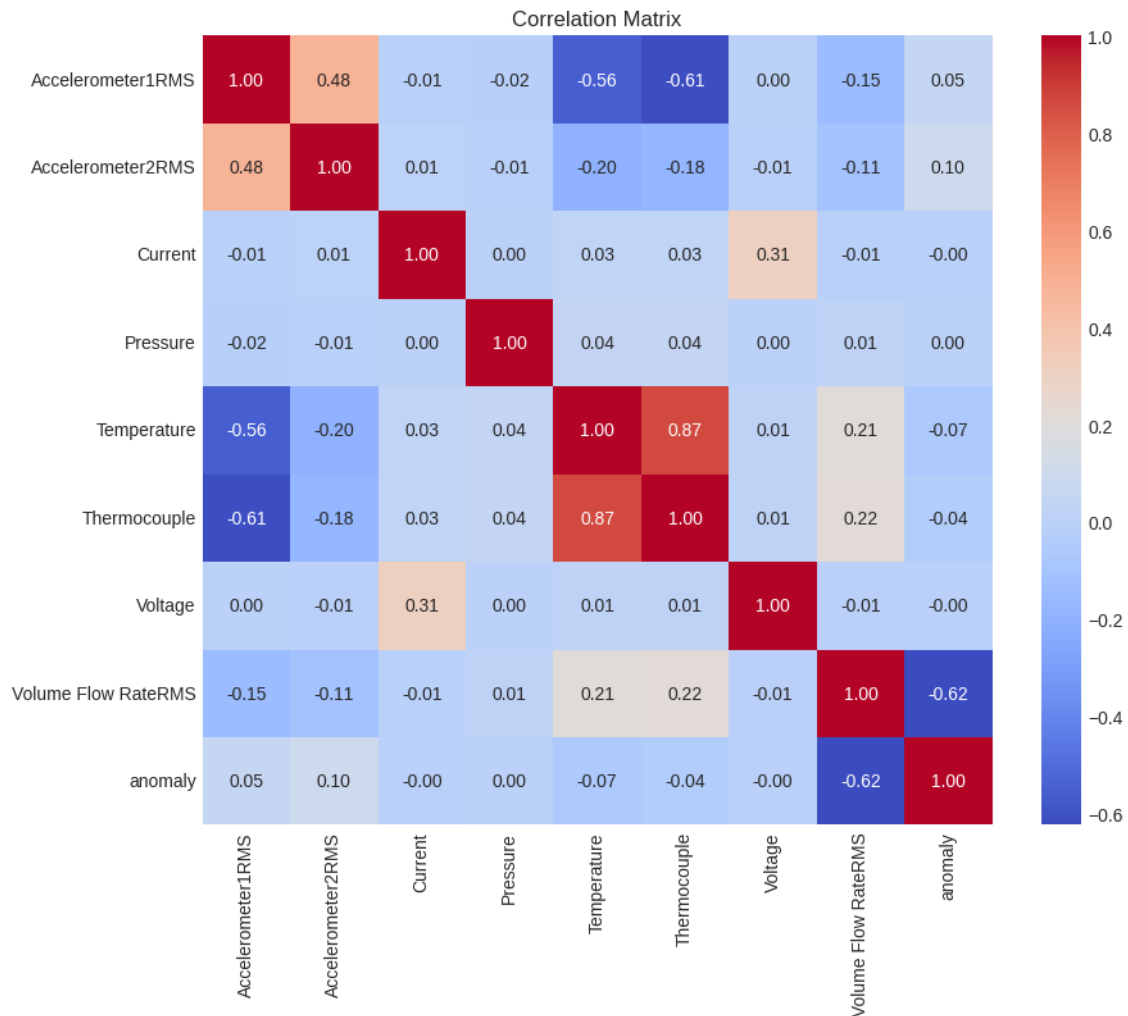


図1 相関ヒートマップ

図1より、anomalyとVolume Flow RateRMSの間には強い負の相関があることが確認できる。これは流体の流れの振動が大きいほど異常であるかどうかの決定に寄与する重要な変数である可能性を示唆している。また、ThermocoupleとTemperatureの間には強い相関が確認できる。これはエンジンの温度の上昇により、循環水の温度も上昇しているからであると考ええる。

2.2. 使用モデル

機械学習の手法は、ランダムフォレスト(Random Forest)およびXGBoost(eXtreme Gradient Boosting)、SVM(Support Vector Machine)を用いた。ランダムフォレスト(Random Forest)は、バギング(Bagging)に基づくアンサンブル学習で、決定木を並列に多数作成し、それぞれの予測結果の平均をとる手法である。XGBoost(eXtreme Gradient

Boosting)は、ブースティングに基づくアンサンブル学習で、前の決定木が誤った部分を次の決定木が重点的に学習・修正するプロセスを繰り返す手法である。SVM(Support Vector Machine)は、教師あり学習に属する手法で、データを最も効果的に分離する境界線を見つけ出すものである。

以下の表2にデータの概要を示す。前処理としてモデルが学習すべきセンサーデータのパターンではない datetime(日時データ)と正解に近いメタデータと考えられる changepoint(変化点フラグ)を削除した。

表2 SKAB データの概要

変数名	count	mean	std	min	max
Accelerometer1RMS	18160	0.027338	0.000527	0.025553	0.029031
Accelerometer2RMS	18160	0.040546	0.001168	0.037339	0.053439
Current	18160	0.974279	0.273464	0.355411	1.662610
Pressure	18160	0.063758	0.258584	-1.257000	1.694350
Temperature	18160	70.207720	2.608980	65.188700	79.889100
Thermocouple	18160	25.029108	0.447720	24.418700	26.104400
Voltage	18160	230.710674	10.817408	203.135000	255.324000
Volume Flow	18160	31.284849	1.991965	22.000000	33.969400
RateRMS					
anomaly	18160	0.347412	0.476161	0.000000	1.000000

SVM のために StandardScaler を用いてデータを標準化し、モデル構築を行った。また、学習データをテストデータの割合が7:3になるようにデータを分割した。

2.3. 評価指標

本レポートでは F1 スコアをモデルの評価指標として用いた。F1 スコアは次の(1)の式で定義される。

$$F1 = \frac{2 * P * R}{P + R} \quad (1)$$

ここで、P を適合率、R を再現率と定義した。F1 スコアは 0~1 の連続値をとり、1 に近いほど精度の良いモデルであることを示す。

2.4. SKAB データに対するモデルの評価結果

以下の表3に XGBoost、Random Forest、SVM(RBF)のそれぞれモデルを構築し、F1 Score(Macro average)で評価を行った。

表 3 モデルの評価結果

モデル名	F1 Score(Macro)
XGBoost	0.949410
Random Forest	0.957996
SVM	0.875358

ランダムフォレスト(Random Forest)と XGBoost(eXtreme Gradient Boosting)がともに F1 スコアが 0.94 以上と非常に高い精度を記録しており、SVM(Support Vector Machine)を大きく上回った。これは SKAB のようなセンサーデータが単純な線形分離では判別できないケースが多くあるためであり、矩形の領域で正常・異常が決まることが多いため、決定木ベースのモデルが適していることを示唆している。

一般に XGBoost(eXtreme Gradient Boosting)はバイアスを減らすのに強く、ランダムフォレスト(Random Forest)は分散を減らすのに強い傾向がある。そのため本解析においてランダムフォレスト(Random Forest)のほうの F1 スコアが高かったということは、今回のデータにはセンサー特有のノイズが含まれており、過学習を抑えやすいランダムフォレスト(Random Forest)のほうが、わずかに汎化性能が高かったことを示唆している。しかし、スコアの差は約 0.008 と小さいため、実用上は両モデルとも同程度の識別能力があると考えられるが、どの変数を用いての予測なのかについては明らかでないため、変数重要度分析でそれを明らかにする。

2.5. SKAB データの変数重要度と SHAP 値の比較

2.5.1. SKAB データを用いた変数重要度

変数重要度とは XGBoost と Random Forest のといった決定木ベースのモデルでは、各特徴量がデータを分割する際にどの程度不純度を減少させたか、あるいは精度向上に寄与したかを定量化したものを指し、モデル全体としてどの変数が学習に大きく貢献したかを示す指標である。

また、線形 SVM では係数の大きさを重要度を測ることができ、非線形カーネルでは変数の直接的な寄与を見ることができないため、Permutation Importance を用いた間接的な重要度の評価することができる。

以下の図 2 に各モデルにおける変数重要度を可視化したグラフを示す。本レポートでは SVM の変数重要度は Permutation Importance を用いた評価方法を採用した。

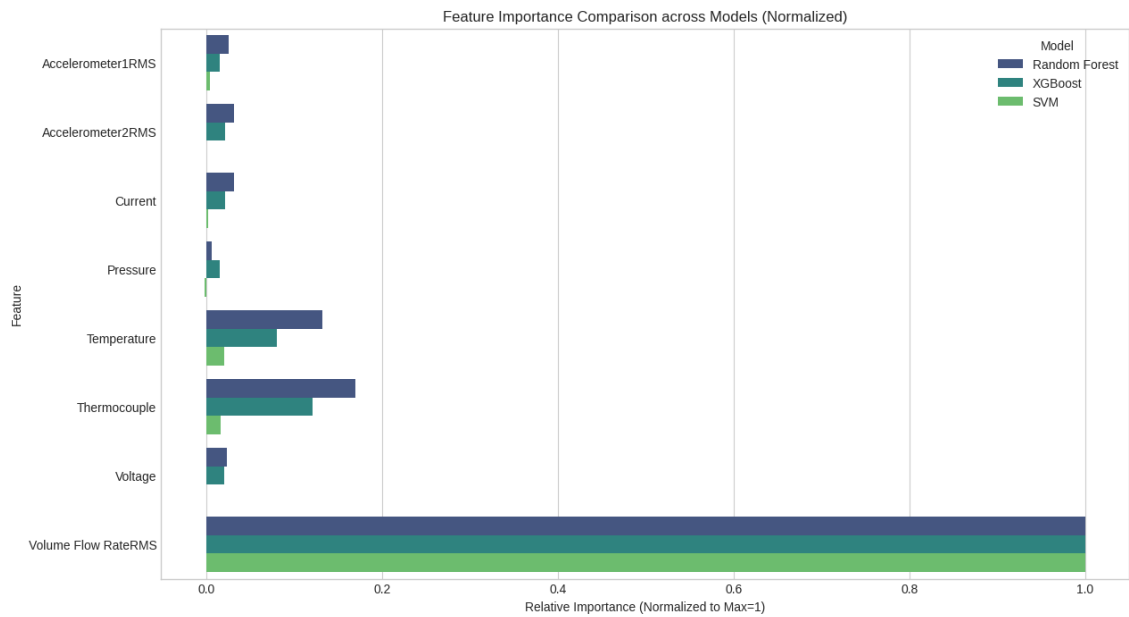


図2 各モデルにおける変数重要度の可視化

図2より、すべてのモデルにおいて Volume Flow RateRMS が最も重要な変数であることが分かる。Normalized Importance が他の変数と比較して圧倒的に重要度が高いことから、異常検知はこの Volume Flow RateRMS の変化にあることが分かる。

各モデルの精度比較において SVM(Support Vector Machine)が精度として低かったのは2番以降の変数を活用できていなかったからであると考えられる。

Accelerometer、Current、Voltage、Pressure はどのモデルにおいても重要度が低く、今回の異常検知タスクにおいては振動や電気系統の変動は以上の主要因ではないことが示唆された。

2.5.2. SKAB データを用いた SHAP 値

SHAP 値とは協力ゲーム理論のシャープレイ値を機械学習に応用した手法であり、ある予測結果に対して、各特徴量がその予測値を平均的な予測値からどれだけ押し上げたかもしくは押し下げたかを算出するものである。

変数重要度とは異なり、変数の値の大小と予測値の上昇・下降の関係や個々のデータポイントごとの寄与まで詳細に分解して解釈できる。

3つのモデルすべてに対して SHAP 値を算出し、各変数が予測値に対してどの程度の影響を与えているかについて検証する。各モデルに対してそれぞれ Water Fall Plot、Bar Plot、Summary Plot の3つのグラフを作成した。最初にランダムフォレスト(Random Forest)について作成したグラフを以下の図3に Water Fall Plot、図4に Bar Plot、図5に Summary Plot を示す。

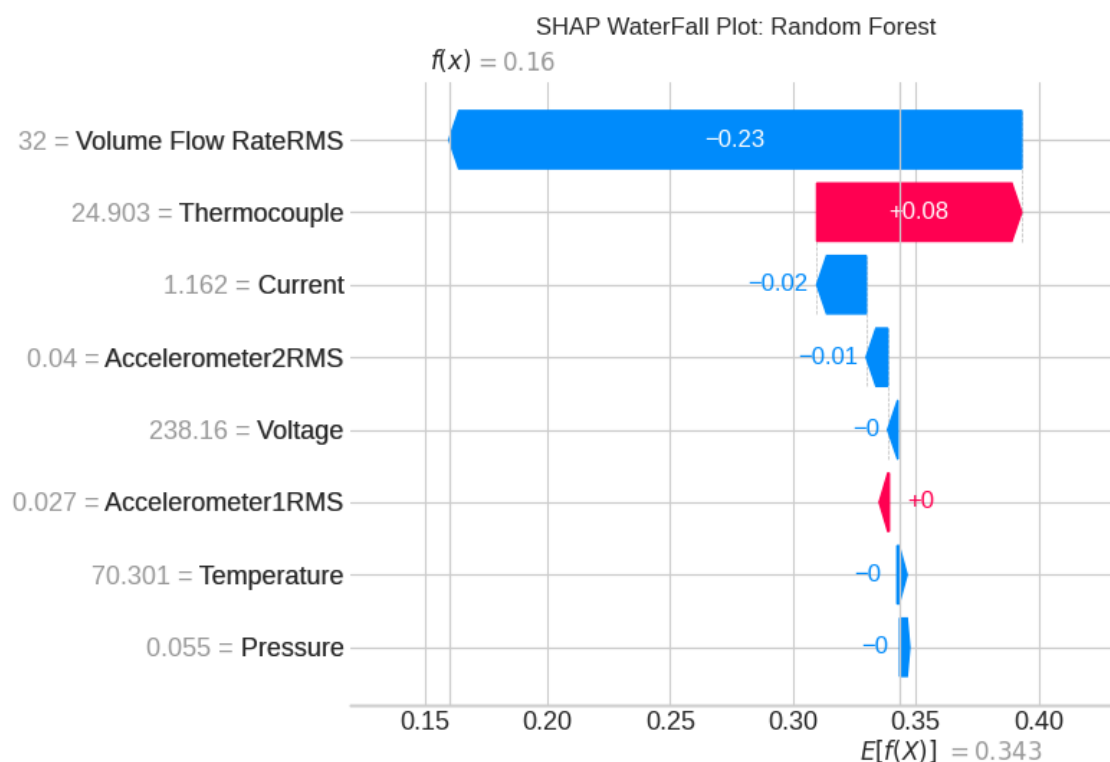


図 3 Water Fall Plot: Random Forest

図 3 より、このサンプルにおいては予測値の平均に比べ、約 0.18 程度低く予想された。スコアを下げた主要因は Volume Flow RateRMS であり、この時の流量は 32 である。これがモデルに対して異常ではないという負の SHAP 値を与えたことが分かる。

一方で Thermocouple は正の SHAP 値を示しており、循環水の温度に関してはわずかに異常傾向が見られたものの、流量の正常性が支配的であったため、最終的には正常と分類されたことがわかる。

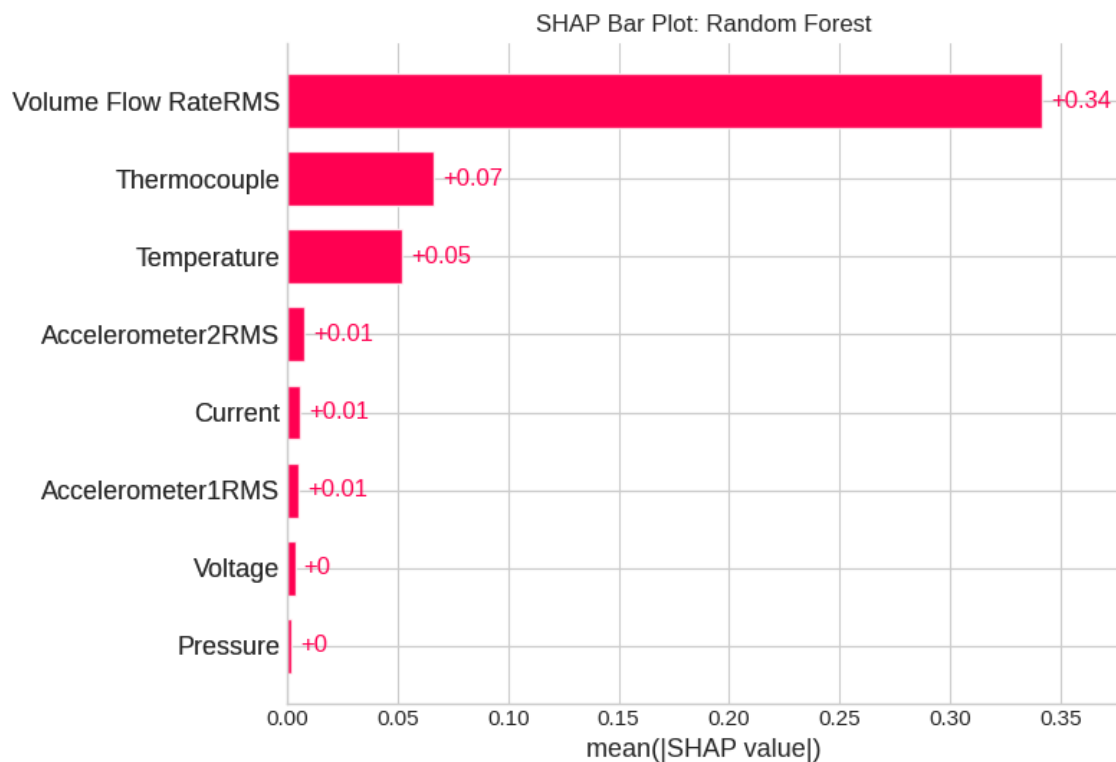


図 4 Bar Plot: Random Forest

図 4 より、Volume Flow RateRMS、Thermocouple、Temperature が特に重要な変数であり、他の変数は補助的、もしくはほとんど利用していないことが分かる。しかし、Thermocouple と Temperature の間には正の相関があるため、重要度が分散してしまった可能性が示唆されている。

モデルが Volume Flow RateRMS に強く依存していることがわかる。そのため、対象システムにおける異常メカニズムは、流体力学的な変動に最も起因しており、モデルの高い精度は流量データに依存しており、温度や振動などの変数は補助的な役割にとどまっている。

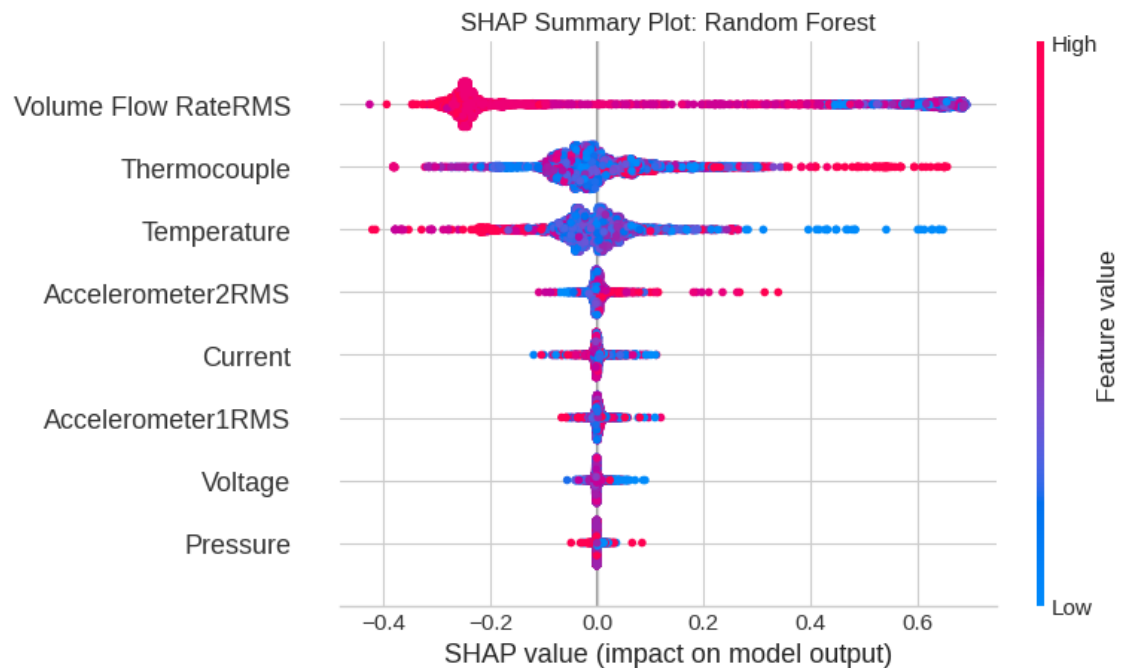


図 5 Summary Plot: Random Forest

図 5 より、ランダムフォレスト (Random Forest) において Volume Flow RateRMS が最も重要な変数であり、赤のプロットが左に偏っていることから負の相関があることが分かる。つまり、詰まりや供給不足によって流量が低下すると異常となることが分かる。

また、Thermocouple と Temperature については他の変数と比べ分布が広がっており、赤と青のプロットが左右両方に存在しているため、温度が高すぎても低すぎても異常であることを示唆している。

他の変数について、わずかに左右に散らばっているが Volume Flow RateRMS と比べると散らばりが小さいため、効果は限定的かノイズであることが分かる。

次に XGBoost(eXtreme Gradient Boosting) について作成したグラフを以下の図 6 に Water Fall Plot、図 7 に Bar Plot、図 8 に Summary Plot を示す。

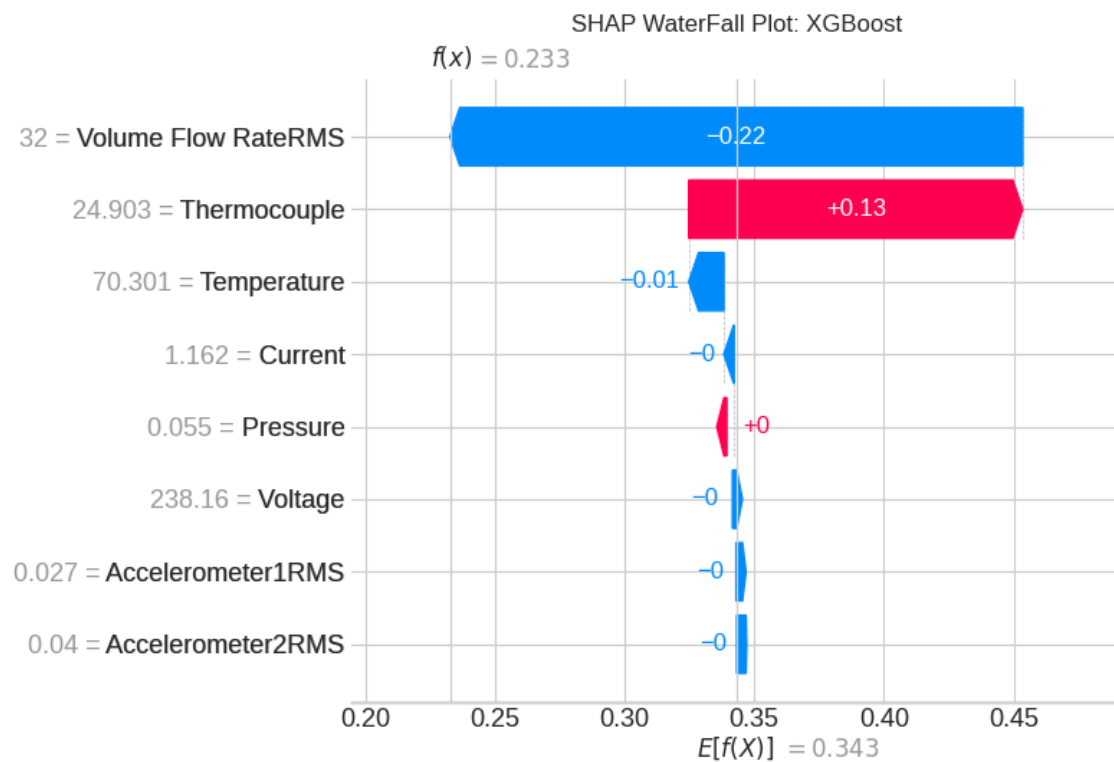


図 6 Water Fall Plot: XGBoost

図 6 より、このサンプルでは全体の予測値の平均よりも 0.11 低い値で予測されている。ランダムフォレスト (Random Forest) と同様に Volume Flow RateRMS が負の寄与をしており、Thermocouple の影響はランダムフォレスト (Random Forest) よりも大きいことがわかる。

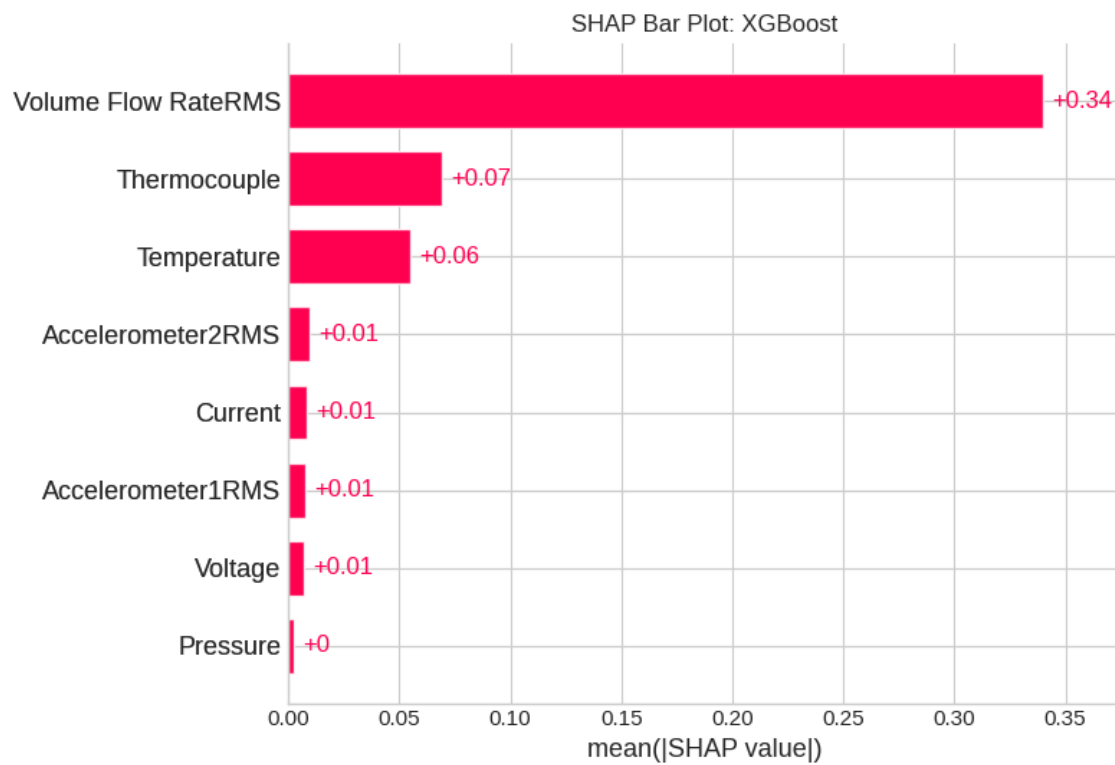


図 7 Bar Plot: XGBoost

図 7 より、XGBoost(eXtreme Gradient Boosting)においても Volume Flow RateRMS が支配的な変数であり、Thermocouple と Temperature といった温度系の変数も同様に上位に来た。また、他の変数についても大きな違いはなかった。

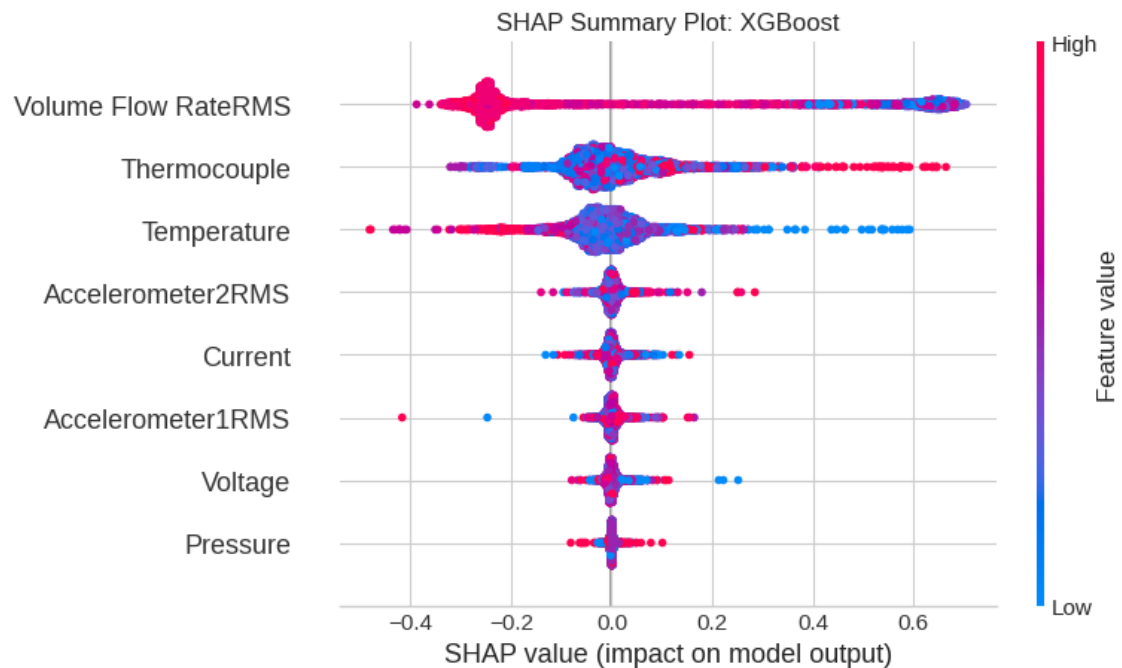


図 8 Summary Plot: XGBoost

図 8 より、XGBoost(eXtreme Gradient Boosting)もランダムフォレスト(Random Forest)と同様に Volume Flow RateRMS、Thermocouple、Temperature が特に重要であり、Volume Flow RateRMS は負の相関があり、Thermocouple と Temperature は高くても低くても異常という結果であることがわかる。

Accelerometer1RMS は、全体的に重要度は低いものの負の方向に他の分布から離れて赤と青の点が一つずつ確認できる。流量の変化による振動の変化といった他の変数との連動が考えられる。

最後に SVM(Support Vector Machine) について作成したグラフを以下の図 9 に Water Fall Plot、図 10 に Bar Plot、図 11 に Summary Plot を示す。

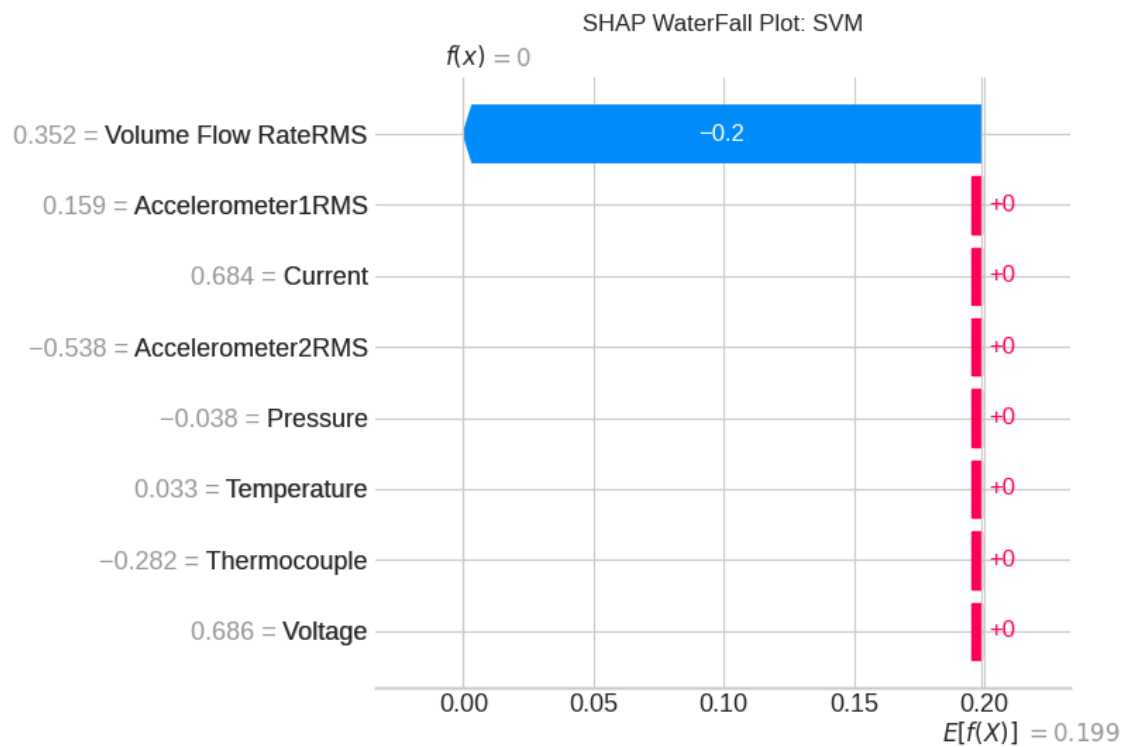


図 9 Water Fall Plot: SVM

図 9 より、このサンプルにおいては全体の予測値の平均に対して 0.2 低い値つまり、確信をもって正常であると判断したことが分かる。これはこのデータ点が決定境界上にある可能性を示唆しており、学習の過程で流量以外の変数をノイズとして扱った可能性を示唆している。

このように流量以外の変数の情報を全く利用していないデータがあることから他のモデルと比較して精度が劣ってしまったと考える。

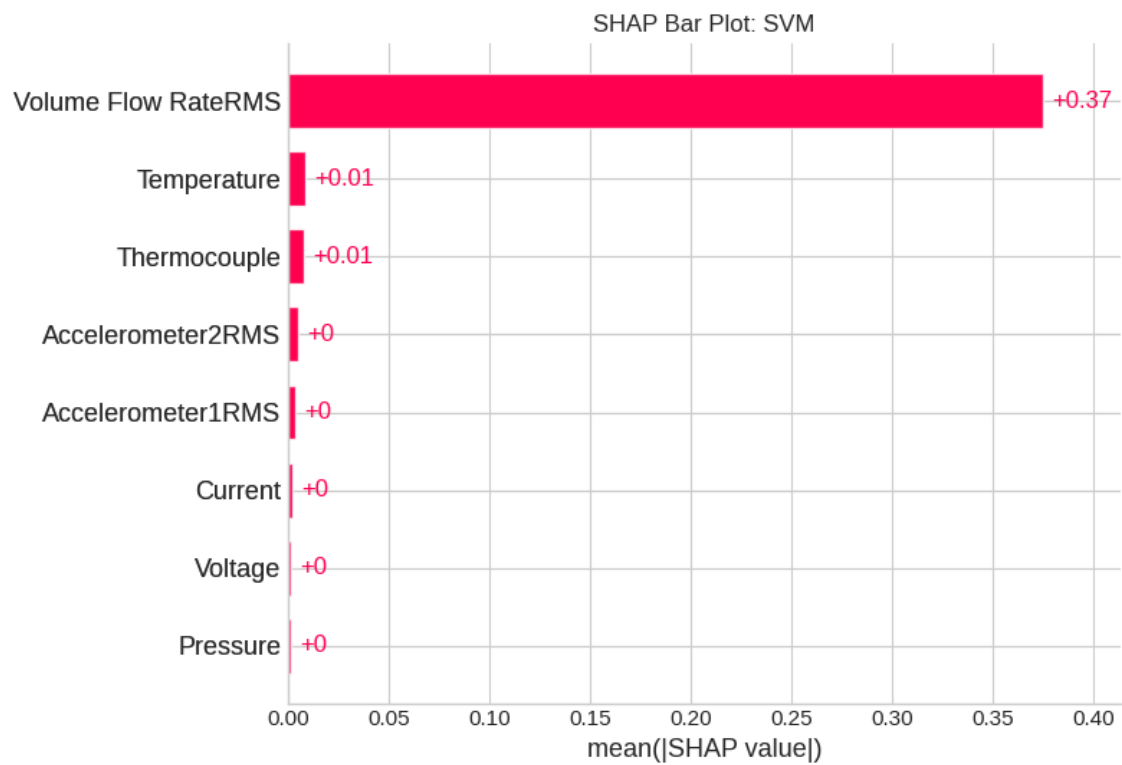


図 10 Bar Plot: SVM

図 10 より、SVM(Support Vector Machine)は特に Volume Flow RateRMS に依存しており、他の変数はほぼノイズとして扱っていることがわかる。

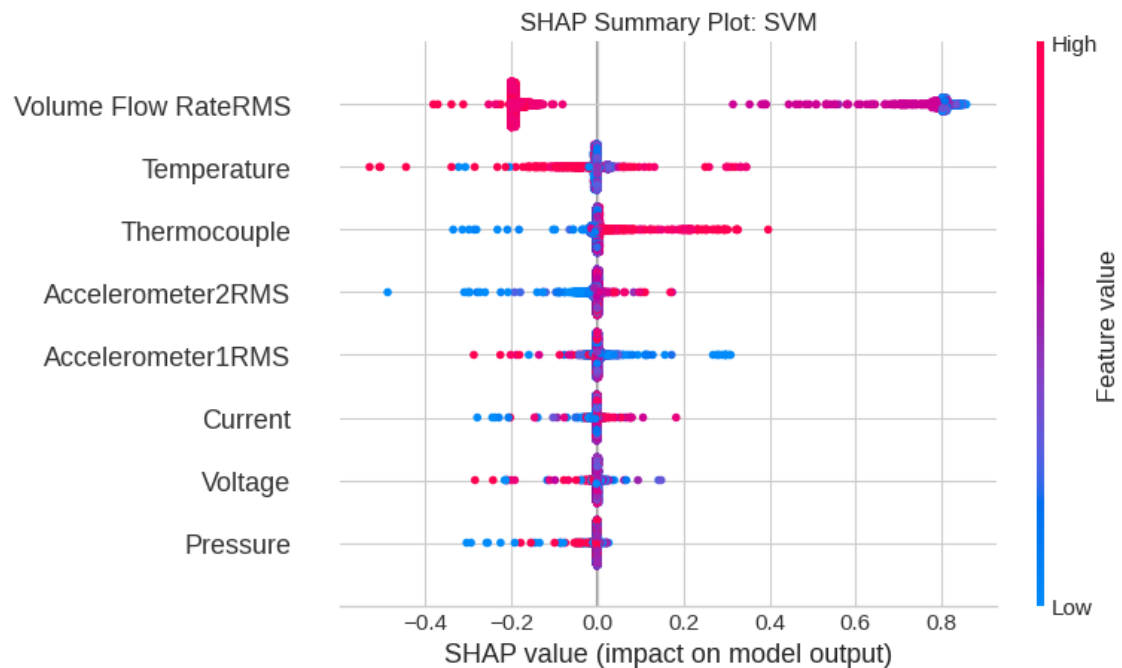


図 11 Summary Plot: SVM

図 11 より、SVM(Support Vector Machine)も同様に Volume Flow RateRMS が最も広く分布しており、負の相関がみられる。他の変数の重要度は著しく低いにもかかわらず分布としてはある程度広がっている。

これは分布としては広がっているが異常かどうかの判断を覆すほどの影響力がないこと、ノイズとしての側面が大きいことから変数重要度としては小さなものとなったと考える。

2.6. SKAB データにおける異常検知モデルの比較考察

すべてのモデルにおいて、Volume Flow RateRMS が最も高い変数重要度を示し、SHAP 値の Summary Plot においても負の相関が確認された。これは対象とするシステムにおける異常の多くが流量の低下であることを強く示唆している。

決定木ベースのモデルに対して SVM(Support Vector Machine)は精度としては劣った。これは多変量データの活用能力にあると考える。SHAP 値解析の結果から決定木ベースのモデルでは Thermocouple や Temperature といった温度に関する変数を判断材料に使っていたのに対し、SVM(Support Vector Machine)では流量が支配的であった。これらは単独での寄与は少ないものの、流量のみでは判断しづらい箇所において判断材料として機能し、誤検知の抑制に寄与したと考える。また、SHAP 図を見ると SVM(Support Vector Machine)は流量に対して非常に鋭敏なのに対し、決定木ベースのモデルではもう少し滑ら

か、あるいは階段状的に反応しており、このような決定強化の形状の違いも原因であると考ええる。

Accelerometer1RMS などは全体的な重要度は低いものの、特定のデータ点において突出した SHAP 値を示した。これは、流量変化を伴わない突発的な異常が発生した際にモデルが振動データをピンポイントで活用している可能性を示唆している。

3. 結論

本レポートでは、SKAB データセットを用いた異常検知モデルの構築を行い、F1 スコアによる精度評価に加え、変数重要度と SHAP 値を用いたモデルの解釈性検証を行った。

本解析において最も優れた性能を示したのはランダムフォレスト(Random Forest)であった。これは、センサーデータ特有のノイズに対する頑健性と変数間の非線形な相互作用を捉える能力が、本データセットに適していたからである。

異常判定の最大要因は Volume Flow RateRMS であることが分かった。また、決定木ベースのモデルにおいては温度変化も判断材料として用いていることが分かった。

4. 使用コード

```
5. import pandas as pd
6. import numpy as np
7. import matplotlib.pyplot as plt
8. import seaborn as sns
9.
10. from sklearn.model_selection import train_test_split,
    GridSearchCV
11. from sklearn.preprocessing import StandardScaler
12. from sklearn.linear_model import LinearRegression
13. from sklearn.tree import DecisionTreeRegressor
14. from sklearn.ensemble import RandomForestRegressor
15. from sklearn.metrics import mean_squared_error, r2_score
16. from sklearn.svm import SVC
17. from sklearn.inspection import permutation_importance
18. from sklearn.metrics import f1_score, classification_report
19.
20. import xgboost as xgb
21.
```



```

22. import shap
23.
24. from sklearn.datasets import fetch_openml
25. plt.style.use('seaborn-v0_8-whitegrid')
26. plt.rcParams['font.family'] = 'sans-serif'
27. df = pd.read_csv('valve1.csv')
28.
29. df.describe().T[['count', 'mean', 'std', 'min', '50%', 'max']]
30. df.info()
31. # 欠損値処理
32. df.dropna()
33.
34. # 外れ値のチェック
35. # IQR 法
36. name = 'Accelerometer1RMS'
37. Q1 = df[name].quantile(0.25)
38. Q3 = df[name].quantile(0.75)
39. IQR = Q3 - Q1
40. lower_bound = Q1 - 3 * IQR
41. upper_bound = Q3 + 3 * IQR
42.
43. outliers = df[(df[name] < lower_bound) | (df[name] >
    upper_bound)]
44. print(f"{name}の外れ値")
45. print(outliers[[name]])
46. print(len(outliers))
47.
48. # 箱ひげ図
49. fig, (ax_box, ax_hist) = plt.subplots(2, 1, sharex=True,
50.                                     gridspec_kw={"height_ratios"
51. : (.15, .85)}),
51.                                     figsize=(10, 6))
52.
53. # 上段:箱ひげ図
54. sns.boxplot(x=df[name], ax=ax_box, color='lightblue',
    flierprops={"marker": "x", "markeredgecolor": "red"})

```

```

55.     ax_box.set(xlabel='') # 上の図の x 軸ラベルは消す
56.
57.     # 下段:ヒストグラム
58.     sns.histplot(df[name], ax=ax_hist, kde=True,
                    color='skyblue')
59.
60.     # タイトルなどの調整
61.     ax_box.set_title(f'Distribution & Outliers of "{name}"',
                        fontsize=14)
62.     plt.show()
63.
64.     # 外れ値の削除
65.     df = df[(df[name] >= lower_bound) & (df[name] <=
        upper_bound)]
66.     target = 'anomaly'
67.
68.     df = df.drop(["datetime", "changeoint"], axis=1)
69.
70.     features = df.drop(columns=[target]).columns.tolist()
71.     # 相関行列の確認(多重共線性のチェック)
72.     plt.figure(figsize=(10, 8))
73.     corr_matrix = df.corr()
74.     sns.heatmap(corr_matrix, annot=True, fmt=".2f",
                    cmap='coolwarm', square=True)
75.     plt.title("Correlation Matrix")
76.     plt.show()
77.     # 変数別にデータを分解
78.     #df_del=df.drop(columns="fixed_acidity")
79.     features = df.drop(columns=[target]).columns.tolist()
80.
81.     y = df[target]
82.     X = df.drop(columns=[target])
83.     # データの分割
84.     X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=0.3, random_state=42)
85.     # 標準化

```

```

86.     scaler = StandardScaler()
87.     X_train_scaled = scaler.fit_transform(X_train)
88.     X_test_scaled = scaler.transform(X_test)
89.     # 結果の保存用辞書
90.     result = {}
91.     models = {}
92.     # モデルの構築
93.     # ランダムフォレスト
94.     rf = RandomForestRegressor(n_estimators=100,
                                random_state=42)
95.     rf.fit(X_train, y_train)
96.     models['Random Forest'] = rf
97.
98.     # XGBoost
99.     xgb_param_grid = {
100.         'n_estimators': [100, 200],
101.         'max_depth': [3, 5, 7],
102.         'learning_rate': [0.01, 0.1]
103.     }
104.     xgb_reg = xgb.XGBRegressor(objective='reg:squarederror',
                                random_state=42)
105.     grid_search = GridSearchCV(xgb_reg, xgb_param_grid, cv=5,
                                scoring='neg_mean_squared_error')
106.     grid_search.fit(X_train, y_train)
107.
108.     best_xgb = grid_search.best_estimator_
109.     models['XGBoost'] = best_xgb
110.     print(f"¥nBest XGBoost Params: {grid_search.best_params_}")
111.
112.     # SVM
113.     svm = SVC(kernel='rbf', probability=True, random_state=42)
114.     svm.fit(X_train_scaled, y_train)
115.     models['SVM'] = svm
116.     # 予測と評価
117.     metrics_data = []
118.

```

```

119. for name, model in models.items():
120.     if name == "SVM":
121.         y_pred = model.predict(X_test_scaled)
122.     else:
123.         y_pred = model.predict(X_test)
124.         # Convert continuous predictions from regressors to
         binary using a threshold (e.g., 0.5)
125.         y_pred = (y_pred >= 0.5).astype(int)
126.
127.     f1score = f1_score(y_test, y_pred, average='macro')
128.
129.     metrics_data.append({'Model': name, 'F1 score': f1score})
130.
131. metrics_df = pd.DataFrame(metrics_data)
132. print(metrics_df)
133. # モデル間の変数重要度 比較分析
134. feature_importance_data = []
135.
136. rf_importance = models['Random Forest'].feature_importances_
137. rf_imp = pd.Series(rf_importance,
                     index=features).sort_values(ascending=True)
138. for name, val in zip(features, rf_importance):
139.     feature_importance_data.append({'Model': 'Random Forest',
                                     'Feature': name, 'Importance': val})
140.
141. xgb_importance = best_xgb.feature_importances_
142. xgb_imp = pd.Series(xgb_importance,
                     index=features).sort_values(ascending=True)
143. for name, val in zip(features, xgb_importance):
144.     feature_importance_data.append({'Model': 'XGBoost',
                                     'Feature': name, 'Importance': val})
145.
146. # SVM
147. svm_result = permutation_importance(svm, X_test_scaled,
                                     y_test, n_repeats=10, random_state=42)
148. svm_importance = svm_result.importances_mean

```

```

149. svm_imp = pd.Series(svm_importance,
    index=features).sort_values(ascending=True)
150. for name, val in zip(features, svm_importance):
151.     feature_importance_data.append({'Model': 'SVM',
    'Feature': name, 'Importance': val})
152.
153. df_importance = pd.DataFrame(feature_importance_data)
154.
155. df_importance['Normalized Importance'] =
    df_importance.groupby('Model')['Importance'].transform(lambda x:
    x / x.max())
156.
157. plt.figure(figsize=(14, 8))
158. sns.barplot(x='Normalized Importance', y='Feature',
    hue='Model', data=df_importance, palette='viridis')
159. plt.title('Feature Importance Comparison across Models
    (Normalized)')
160. plt.xlabel('Relative Importance (Normalized to Max=1)')
161. plt.show()
162.
163. print(df_importance)
164. # SVM (KernelExplainerを使用 - 計算コストが高いためデータの一部を使用)
165. # 背景データとして訓練データの一部(kmeans で要約)を使用
166. X_train_summary = shap.kmeans(X_train_scaled, 10)
167.
168. for name, model in models.items():
169.     print(f"¥nAnalyzing: {name} ...")
170.
171.     plt.figure() # 新しい描画領域を作成
172.
173.     if name == "SVM":
174.         # explainer = shap.KernelExplainer(model.predict,
    X_train_scaled)
175.         explainer = shap.KernelExplainer(model.predict,
    X_train_summary)
176.         shap_values = explainer(X_test_scaled)

```

```
177.         shap_values.feature_names = features
178.     else:
179.         # 決定木ベースモデル用の Explainer (TreeExplainer)
180.         explainer = shap.TreeExplainer(model)
181.         shap_values = explainer(X_test, check_additivity=False)
182.         shap_values.feature_names = features
183.
184.     plt.title(f"SHAP WaterFall Plot: {name}")
185.     shap.plots.waterfall(shap_values[0], show=False)
186.
187.     plt.tight_layout()
188.     plt.show()
189.
190.     plt.title(f"SHAP Bar Plot: {name}")
191.     shap.plots.bar(shap_values, show=False)
192.
193.     plt.tight_layout()
194.     plt.show()
195.
196.     plt.title(f"SHAP Summary Plot: {name}")
197.     shap.summary_plot(shap_values, show=False)
198.
199.     plt.tight_layout()
200.     plt.show()
```

2026 年 1 月 10 日

科目名：生産管理

担当教授：大草 孝介

ワインのクオリティ (quality) の予測モデル構築と SHAP 値による要因分析の精度評価

1. 序論

1.1. ワインの評価方法

本レポートで扱う「ワインの品質(quality)」は、専門家による官能評価に基づいたものであり、評価値は 0~10 までの整数値でスコアリングされており、味覚、嗅覚、視覚(色味)などの複合的な要素が反映されている主観的な指標である一般的に数値が高いほうが高品質である。

1.2. 背景

ワイン産業において、品質管理とグレーディング(格付け)は製品価格やブランド価値を決定する重要なプロセスである。しかし、官能評価は専門家の育成に時間がかかり、個人の体調や主観に左右される可能性があるため、時間的・金銭的成本と安定性の面で課題がある。そこで、物理化学的な成分データから品質を客観的に推定できれば、製造プロセスの効率化や品質の安定化に寄与すると考える。

1.3. 問題設定

ワイン製造における品質の人的判断には時間・金銭的なコストがかかり、安定性に欠ける可能性があるという問題を抱えている。そこで、本レポートでは、赤ワインの理化学的特性(アルコール度数、pH など)を説明変数とし、専門家による官能評価値(0~10)を説明変数とする回帰問題について考える。

1.4. 目的

本レポートでは赤ワインの理化学的特性から官能評価値をどの程度予測できるかを検証し、どの変数が予測に有効かについて検討する。

予測精度の比較には RMSE と R2 を比較し、最適な予測手法を特定する。また、各機械学習モデルにおける変数重要度、SHAP 値を算出し、それらを用いてワインの評価に強く影響を与える成分を特定し、その影響の傾向を考察する。

機械学習のモデルとして決定木、ランダムフォレスト(Random Forest)、XGBoost(eXtreme Gradient Boosting)、SVM(Support Vector Machine)を用いて上記 2 点

について検証考察を行うことを目的とした。

2. 分析手法

2.1. データセットの構築

本レポートでは、Python ライブラリ sklearn にあるデータセット wine-quality-red を用いてデータを取得した。欠損値を含むデータは除外し、モデル構築用のデータセットを作成した。

2.2. 変数の選定

本レポートで扱うデータセットにおいて、ワインの品質を表す変数は class で、データタイプが category である。そのため、変数名を「quality」に変更し、データタイプを整数 (int)に変更した。ワインの品質を示す変数(quality)を除いた wine-quality-red に含まれている変数の説明を以下の表 1 に示す。

表 1 変数の説明

変数名	説明
fixed_acidity	固定酸
volatile_acidity	揮発酸
citric_acid	クエン酸
residuak_suger	残留糖
chlorides	塩化物
free_sulfur_dioxide	遊里亜硫酸
total_sulfur_dioxide	総亜硫酸
density	密度
pH	水素イオン指数
sulphates	硫酸塩
alcohol	アルコール度数

変数の選定のために、相関分析を実施する。各変数間の相関行列をヒートマップで表したものを以下の図 1 に示す。

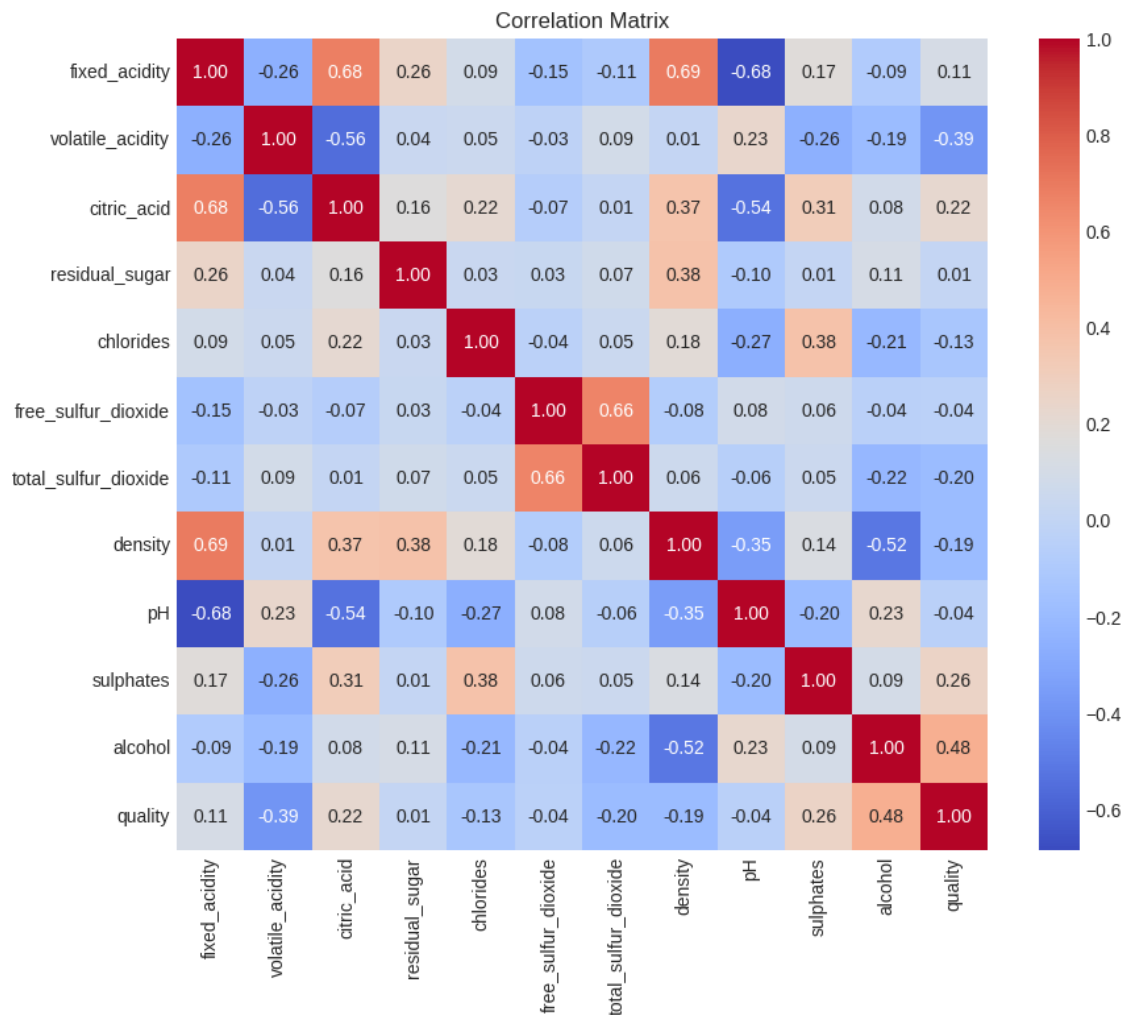


図 1 相関ヒートマップ

図 1 より、ワインの品質(quality)と揮発酸(volatile acidity)の間には負の相関がみられ、ワインの品質(quality)とアルコール度数(alcohol)の間には比較的強い正の相関が確認できる。これは、アルコール度数の多寡が品質の決定に寄与する重要な変数である可能性を示唆している。本レポートではこの可能性についても検討する。

また、固定酸(fixed acidity)と密度(density)の間には強い正の相関がみられる。そこで本解析では多重共線性の対策として、ワインの品質(quality)との相関がより小さい固定酸(fixed acidity)を削除し、解析を行う。

2.3. 使用モデル

機械学習の手法は、決定木、ランダムフォレスト(Random Forest)およびXGBoost(eXtreme Gradient Boosting)、SVM(Support Vector Machine)を用いた。決定木は、条件分岐を繰り返すことによってサンプルを分類する機械学習の手法である。ランダムフォレスト(Random Forest)は、バギング(Bagging)に基づくアンサンブル学習で、決定木を並列に多数作成し、それぞれの予測結果の平均をとる手法である。XGBoost(eXtreme Gradient Boosting)は、ブースティングに基づくアンサンブル学習で、前の決定木が誤った部分を次の決定木が重点的に学習・修正するプロセスを繰り返す手法である。SVM(Support Vector Machine)は、教師あり学習に属する手法で、データを最も効果的に分離する境界線を見つけ出すものである。

2.4. 評価指標

本レポートでは二乗平均平方誤差(RMSE, Root Mean Squared Error)と決定係数(R^2 , Coefficient of Determination)をモデルの評価指標として用いた。それぞれ次の(2)、(3)の式で定義される。

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3)$$

ここで、 n はデータ数、 y_i は実測値、 $\hat{y}_i$ は予測値、 $\bar{y}$ は実測値の平均と定義した。RMSE は 0 に近づくほど精度がよく、 R^2 は 1 に近づくほど精度がよい。

3. 予測精度の比較

3.1. データの概要

以下の表 2 にデータの概要を示す。本解析で用いたデータには欠損値がなかったため、データの加工はせずに解析を行う。

表 2 データの概要

	count	mean	std	min	50%	max
volatile_acidity	1511.0	0.527118	0.179265	0.12000	0.52000	1.5800
citric_acid	1511.0	0.266817	0.193244	0.00000	0.25000	1.0000
residual_sugar	1511.0	2.267042	0.600347	0.90000	2.20000	4.7000
chlorides	1511.0	0.087230	0.047583	0.01200	0.07900	0.6110
free_sulfur_dioxide	1511.0	15.525480	9.843923	1.00000	13.00000	57.0000
total_sulfur_dioxide	1511.0	45.068167	30.765095	6.00000	37.00000	165.0000
density	1511.0	0.996652	0.001823	0.99007	0.99668	1.0032
pH	1511.0	3.312369	0.155839	2.74000	3.31000	4.0100
sulphates	1511.0	0.657690	0.171045	0.33000	0.62000	2.0000
alcohol	1511.0	10.411372	1.055826	8.40000	10.10000	14.0000
quality	1511.0	5.631370	0.798041	3.00000	6.00000	8.0000

ここで、外れ値の確認とデータの確認するために残留糖(residual sugar)のデータを用いて箱ひげ図とヒストグラムを作成し、データの特性について確認する。以下の図 2 に箱ひげ図とヒストグラムを示す。

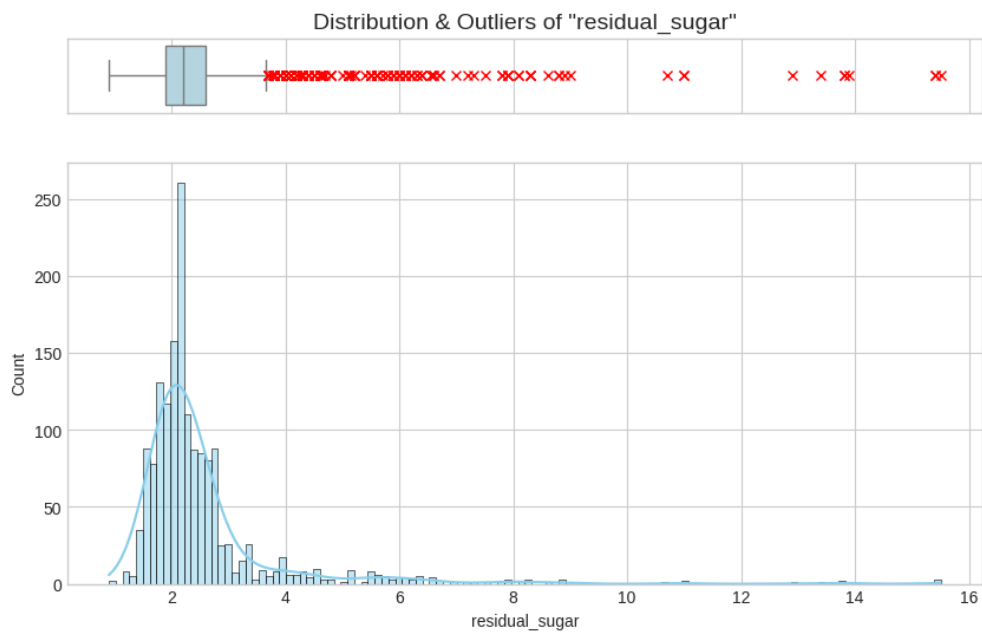


図 2 箱ひげ図とヒストグラム

図2の箱ひげ図より外れ値と思われるデータが複数あると考えられる。しかし、ヒストグラムに着目するとデータに正規性がないことが考えられる。そこで、本解析では説明変数の正規性は仮定せず、データ本来の分布特性を維持するために外れ値の除去処理は行わず、すべてのデータを使用した。

3.2. モデル精度の比較

モデルの学習と評価のために、全データを学習用データ 80%、テスト用データ 20%に分割した。評価はテスト用データに対して行われた。

以下の表2に構築した5つのモデルに対し、テストデータを用いて予測精度を検証した結果を示す。

表2 予測精度の比較

モデル	RMSE	R2_Score
決定木	0.663789	0.288884
ランダムフォレスト	0.546100	0.518690
XGBoost	0.562612	0.489145
SVM	0.579514	0.457989

表2より、ランダムフォレスト(Random Forest)が最も高い精度を示した。ランダムフォレスト(Random Forest)やXGBoost(eXtreme Gradient Boosting)といった決定木ベースのアンサンブル学習モデルが単体の決定木やSVM(Support Vector Machine)よりも高い精度を示した。

SVM(Support Vector Machine)は決定木よりは高い精度を示したが、決定木ベースのアンサンブル学習モデルには及ばなかった。

最も精度の高いモデルでもR2スコアが0.52程度にとどまっていることがわかる。

3.3. 考察

決定木ベースのアンサンブル学習モデルが高い精度を示したのは複数のモデルを組み合わせることで過学習を防ぎつつ、データ内の複雑な非線形な関係を捉えられているためであると考えられる。

SVM(Support Vector Machine)が単体の決定木よりは精度がよかったが他二つには及ばなかったのは本解析で用いたデータセットが決定木系アンサンブルの手法に適していることを示唆している。しかし、変数の正規化やパラメータチューニングによる精度の改善の

余地はあると考える。

本解析によって得た R2 スコアは最大でも 0.52 程度であった。これはワインの品質 (quality) の変動の約半分は今回説明変数として採用した理化学的特性では説明できていないことを意味している。官能評価には個人の主観的な好みや体調、測定していない香り成分、外観の色合いなどが影響しているからであると考ええる。

4. 要因分析

4.1. 変数重要度 (Feature Importance) による分析

本解析で用いる 4 つの機械学習モデルについて、それぞれの変数が評価に寄与しているかを解析する。以下の図 3 に変数重要度を描画したグラフを示す。

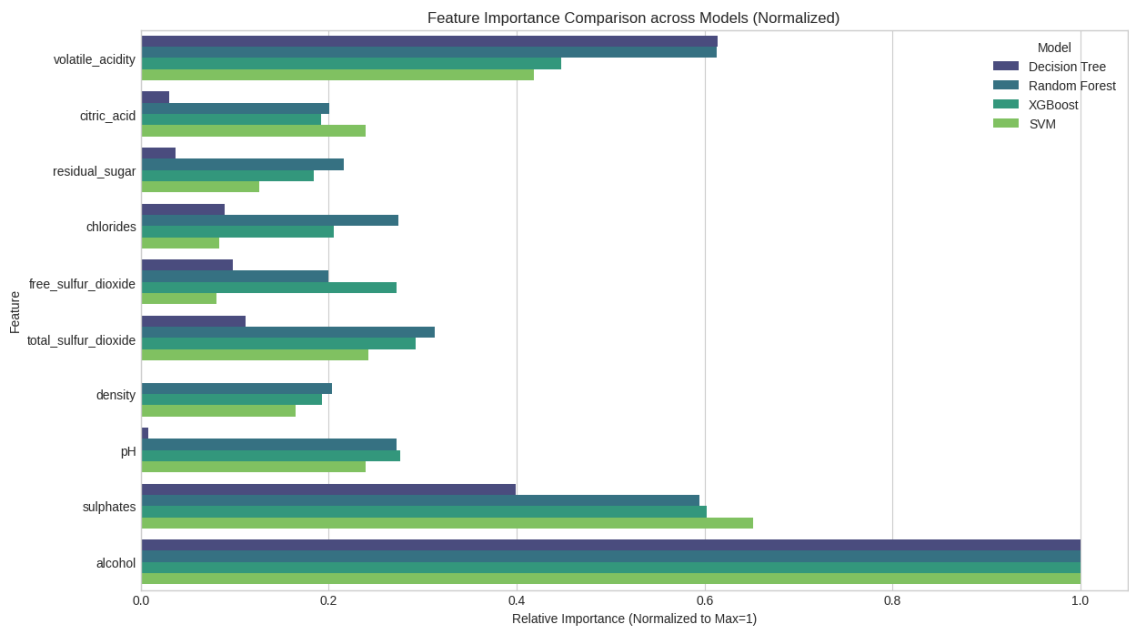


図 3 変数重要度グラフ

図 3 より全てのモデルにおいてアルコール度数(alcohol)が最も重要な変数として特定された。アルゴリズムの違いに関わらずこの結果が一貫していることから、アルコール度数(alcohol)がワインの評価を決定づける支配的な要因であることが分かる。

次いで、揮発酸(volatile acidity)と硫酸塩(sulphates)がすべてのモデルで上位に入っていることがわかる。決定木とランダムフォレスト(Random Forest)では不快臭の原因である揮発酸(volatile acidity)の重要度が比較的高く、XGBoost(eXtreme Gradient Boosting)と SVM(Support Vector Machine)では酸化防止剤や品質保持に関連する硫化塩(sulphates)の

需要度が揮発酸(volatile acidity)の重要度を上回った。

総亜硫酸(total sulfur dioxide)の値は第 4 の変数として挙げられるが、密度(density)や pH は決定木では重要度がほぼ 0 となるなど予測への寄与が限定的な変数であることが分かる。

4.2. SHAP 値による分析

4 つのモデルすべてに対して SHAP 値を算出し、各変数が予測値に対してどの程度の影響を与えているかについて検証する。各モデルに対してそれぞれ Water Fall Plot、Bar Plot、Summary Plot の 3 つのグラフを作成した。最初に決定木について作成したグラフを以下の図 4 に Water Fall Plot、図 5 に Bar Plot、図 6 に Summary Plot を示す。

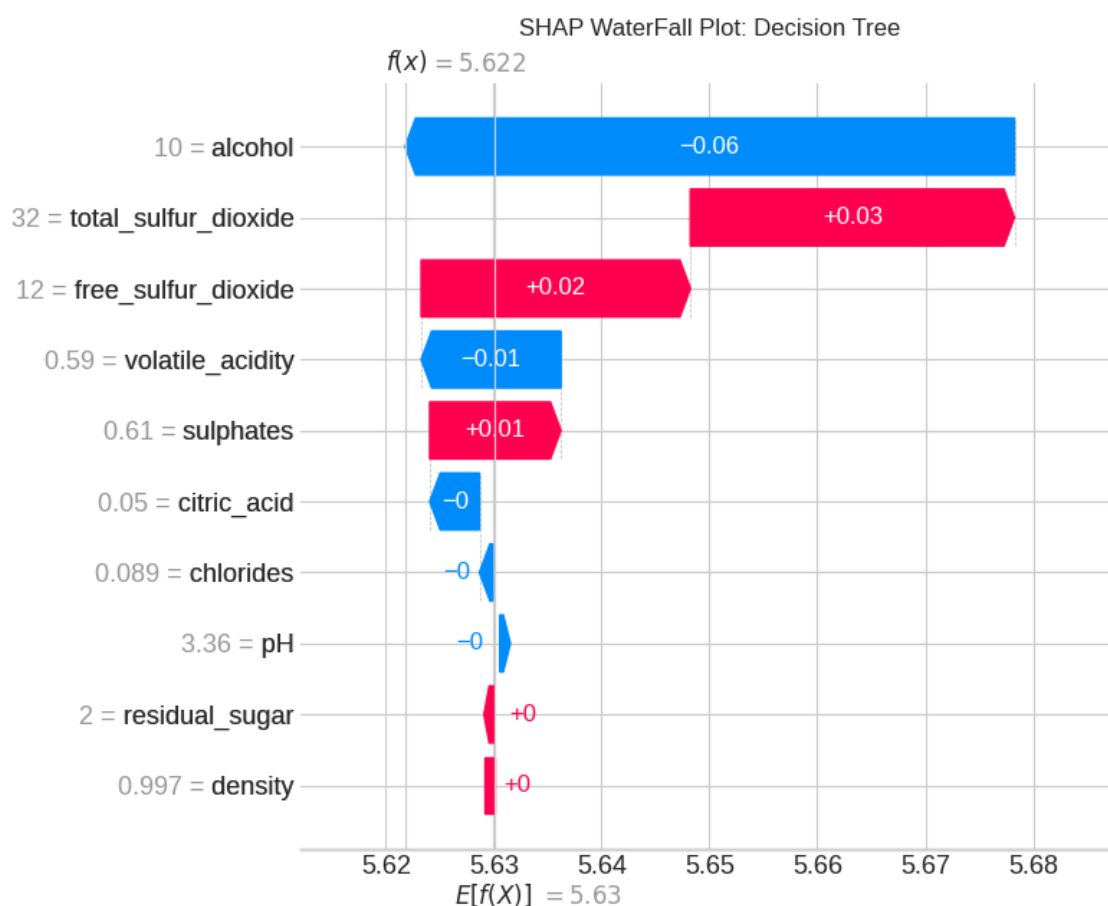


図 4 Water Fall Plot: 決定木

図 4 よりアルコール度数(alcohol)が品質を下げる方向に働いている(=-0.06)が硫酸系の変数により部分的に補正されたことによって予測期待値(=5.63)に対してやや低い予測値

(=5.622)となった。

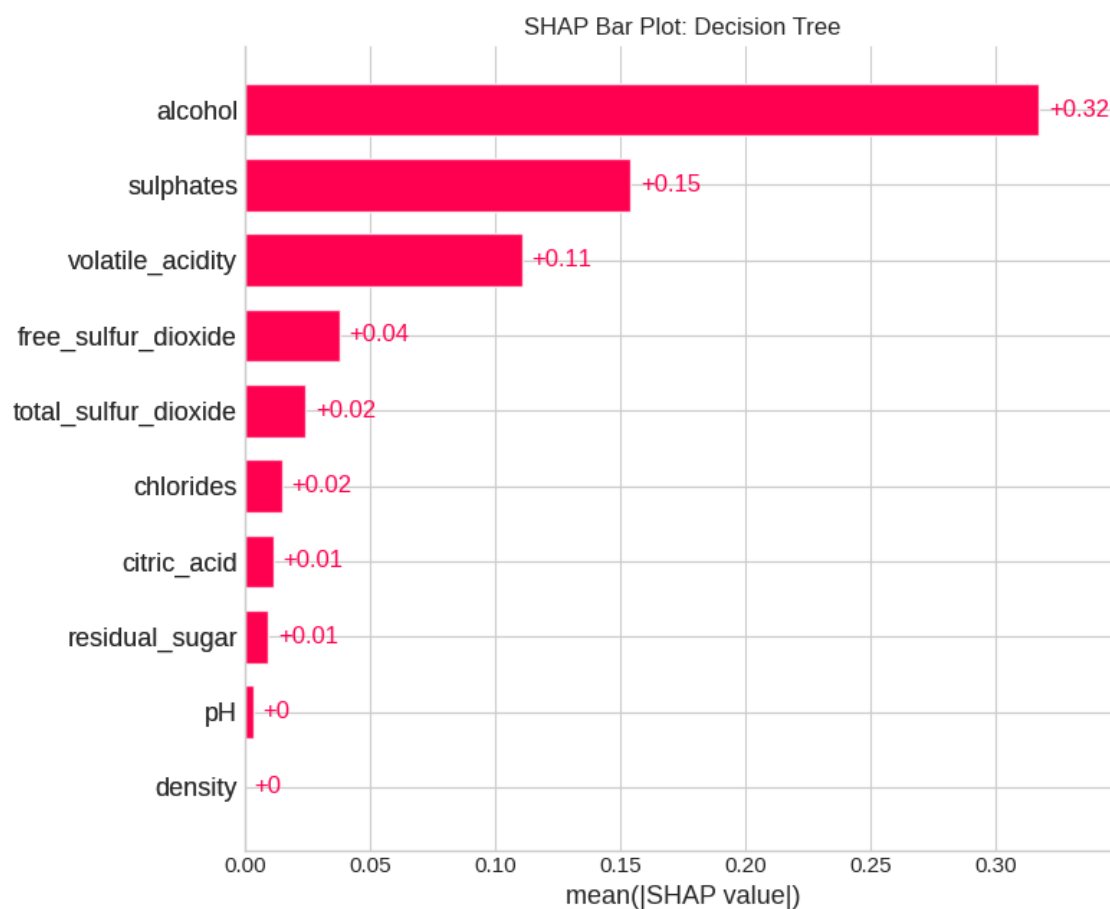


図 5 Bar Plot: 決定木

図 5 より、アルコール度数(alcohol)、硫酸塩(sulphates)、揮発酸(volatile acidity)が特に重要な変数であり、他の変数は補助的、もしくはほとんど利用していないことが分かる。

モデルがアルコール度数(alcohol)に強く依存していることが分かる。そのため、単一木によるバイアスや過学習の可能性について留意する必要がある。また、重要度が急激に落ちているということは分岐構造が比較的単純で少数の変数に依存していることも分かる。

Bar Plot は Water Fall Plot と異なり、モデル全体での重要度を示したものである。そのため、アルコール度数(alcohol)が重要であることはわかるが予測に対して効く方向はサンプル次第であることに留意が必要である。

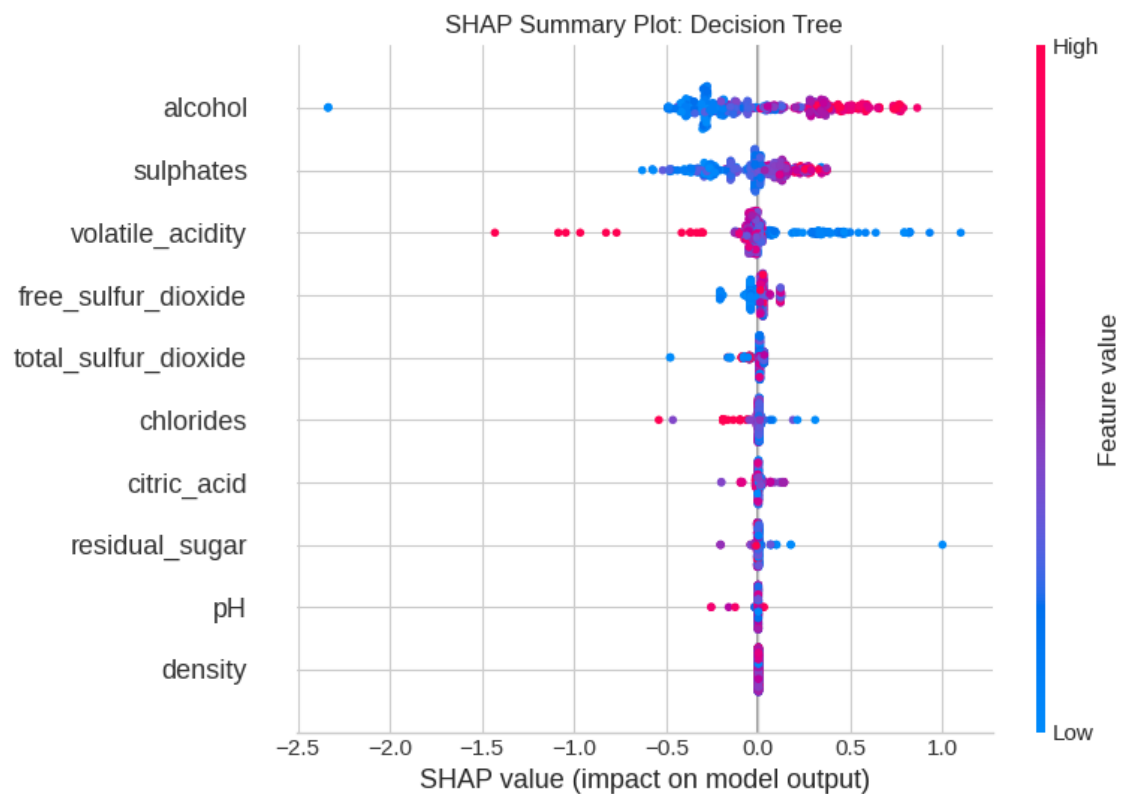


図 6 Summary Plot: 決定木

図 6 より、アルコール度数(alcohol)が最も重要な変数であることが分かる。また、赤のプロットが右側に集中しているため、アルコール度数(alcohol)が高いほど品質(quality)も高くなる傾向があることが分かる。また、一部極端な負の点がある。これより、決定木の特定分岐で強いペナルティを与えていることが分かる。

この決定木モデルは、アルコール度数(alcohol)、硫酸塩(sulphates)、揮発酸(volatile acidity)を主軸に、明確な閾値ルールで品質(quality)を上下させており、それ以外の変数は限定的にしか利用されていないことが分かる。

次にランダムフォレスト(Random Forest)について作成したグラフを以下の図 7 に Water Fall Plot、図 8 に Bar Plot、図 9 に Summary Plot を示す。

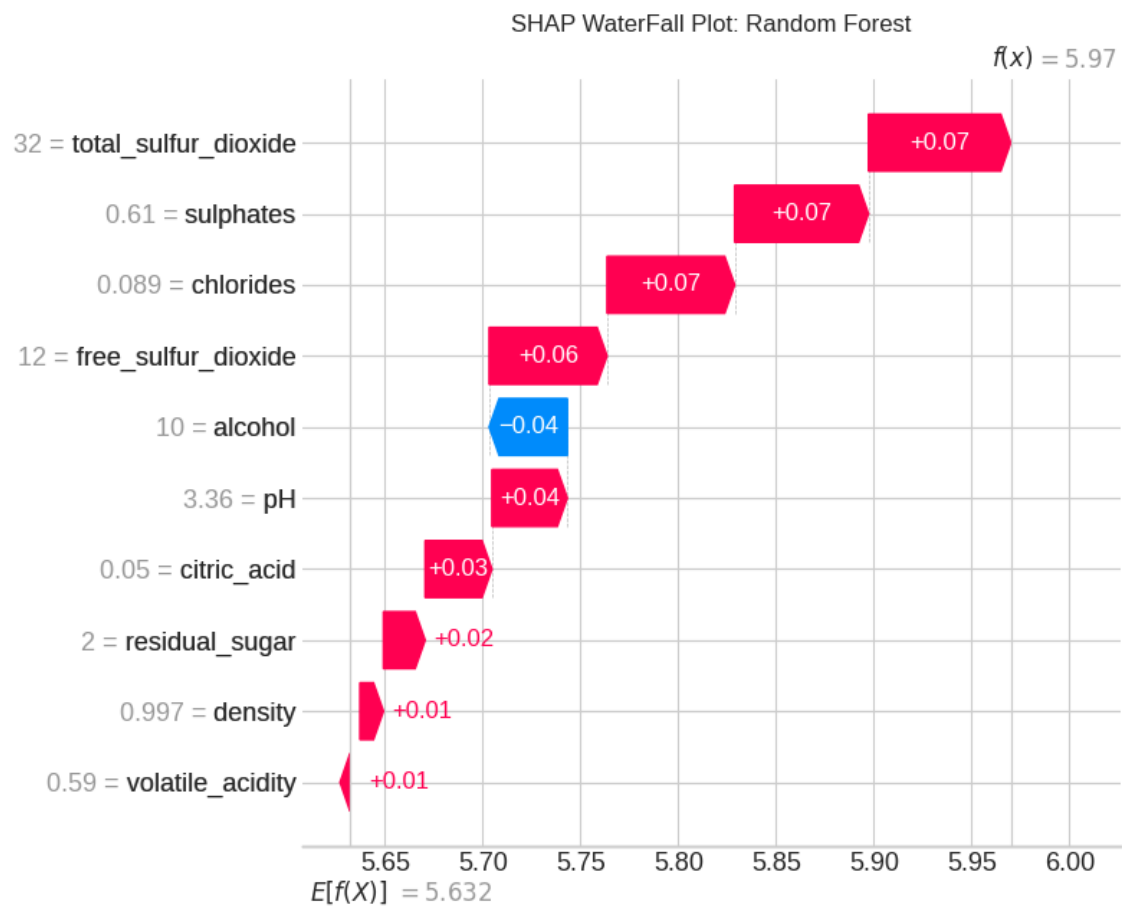


図 7 Water Fall Plot: Random Forest

図 7 より、このサンプルではモデル全体の平均予測値に対して 0.34 程度大きな値が予測された。決定木のものと比較すると多くの変数が小さい寄与を積み重ねて予測していることが分かる。

また、アルコール度数(alcohol)は決定木のもののほど負の効果が大きくないことも分かる。

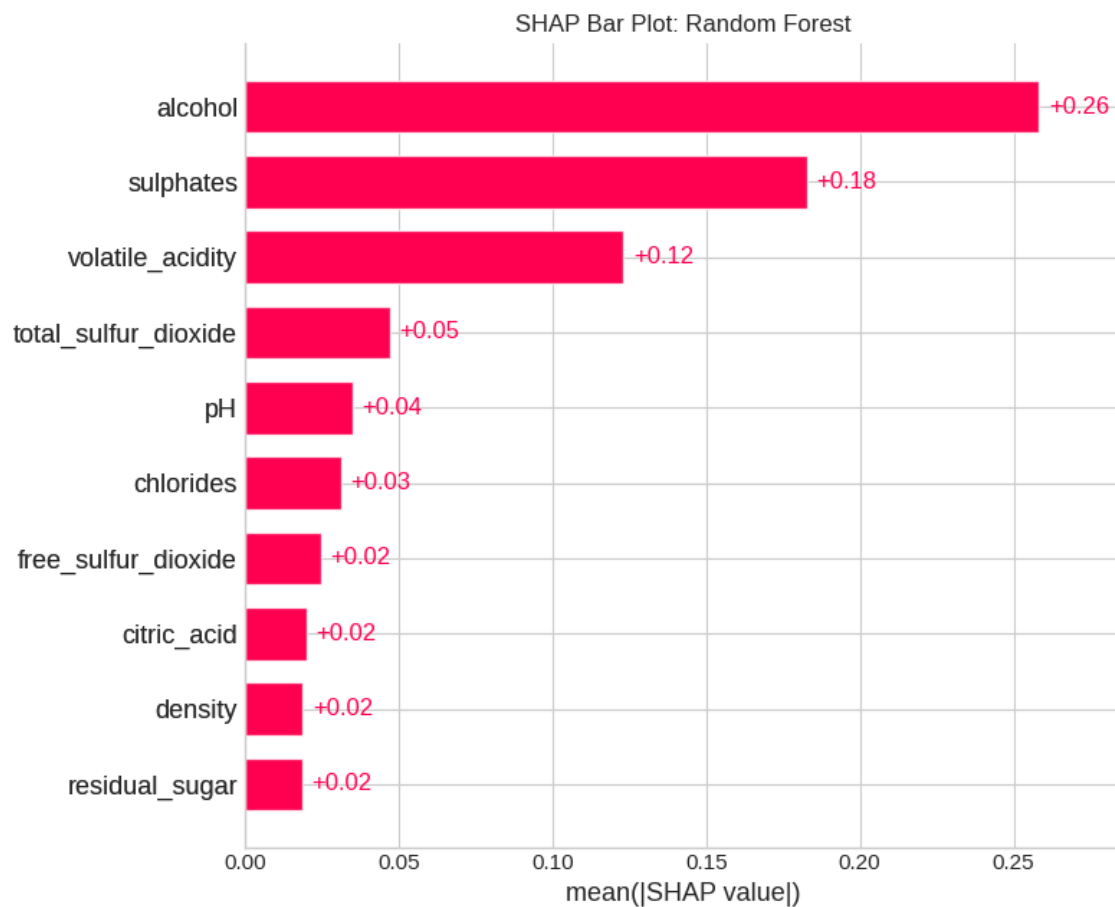


図 8 Bar Plot: Random Forest

図 8 より、ランダムフォレスト(Random Forest)においてもアルコール度数(alcohol)、硫酸塩(sulphates)、揮発酸(volatile acidity)が依然として重要な変数であることが分かる。しかし、変数重要度は 0.32 から 0.26 に減少しており、単一特徴への依存が緩和されたことが分かる。

pH は決定木ではほぼ 0 だったのに対し、0.04 と有効な変数に変わっており、多様な木の中で条件次第では有効な分岐に使われたことが分かる。

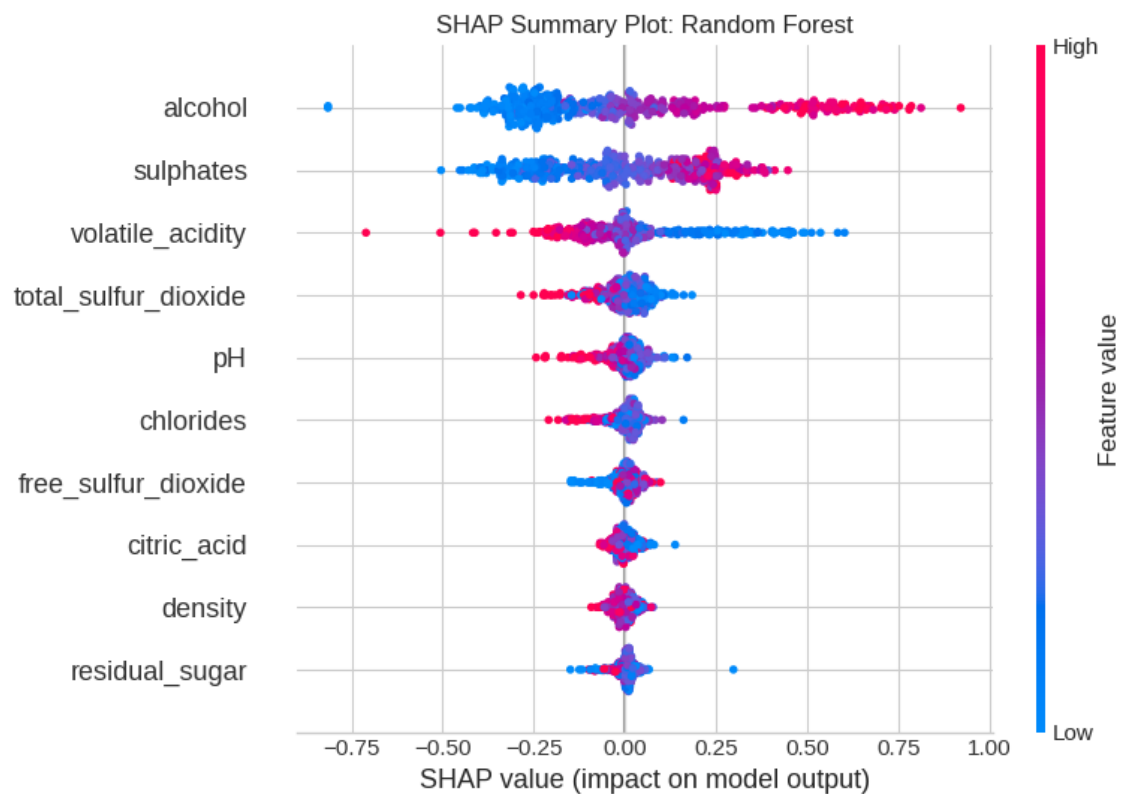


図 9 Summary Plot: Random Forest

図 9 より、ランダムフォレスト(Random Forest)においてもアルコール度数(alcohol)が最も重要な変数であり、赤のプロットが右に偏っていることから正の相関があると考えられる。

ランダムフォレスト(Random Forest)モデルは、主要な変数の影響方向を安定して学習しつつ、単一決定木よりもなめらかで信頼性の高い説明構造を持っていることが分かる。

次に XGBoost(eXtreme Gradient Boosting) について作成したグラフを以下の図 10 に Water Fall Plot、図 11 に Bar Plot、図 12 に Summary Plot を示す。

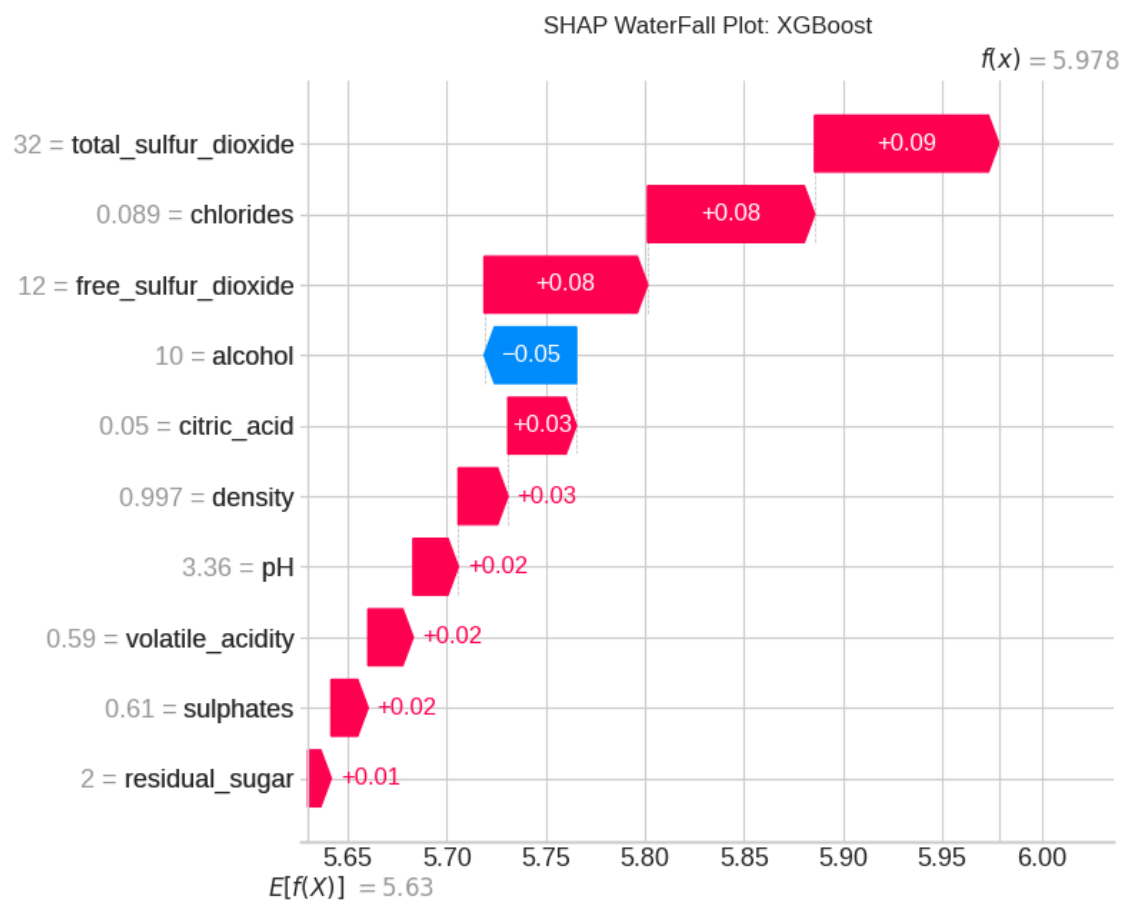


図 10 Water Fall Plot: XGBoost

図 10 より、このサンプルでは全体の予測値の平均よりも 0.35 程度大きな値で予測されている。アルコール度数(alcohol)以外の多くの特徴量で正の寄与が見られる。決定木やランダムフォレスト(Random Forest)と比べると小さな寄与が複数の変数によって加算されていることが分かる。

また、硫黄・塩化物の重要性が強調されたことがわかる。特に総亜硫酸(total sulfur dioxide)、遊里亜硫酸(free sulfur dioxide)、塩化物を組み合わせで評価していることが分かる。

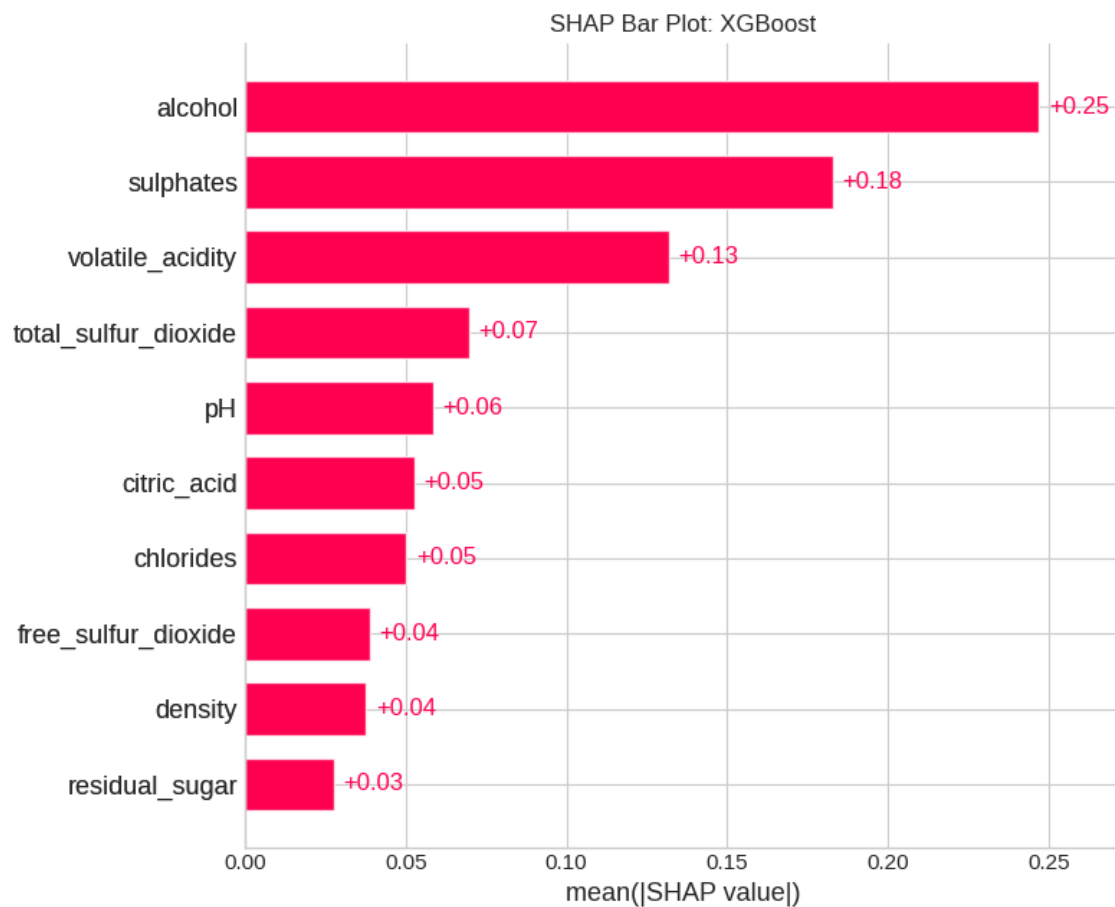


図 11 Bar Plot: XGBoost

図 11 より、XGBoost(eXtreme Gradient Boosting)においてもアルコール度数(alcohol)が最も重要な変数であったことが分かる。硫酸塩(sulphates)、揮発酸(volatile acidity)についても同様に重要な変数であることがわかる。

ランダムフォレスト(Random Forest)と比べると、上位 3 変数以外の変数の重要度が増しており、より均等に変数を予測に用いていることが分かる。このことより、XGBoost(eXtreme Gradient Boosting)モデルは、主要変数への依存を抑えながら、多数の理化学的特性をバランスよく利用することで比較的良好な精度で予測できていたことがわかる。

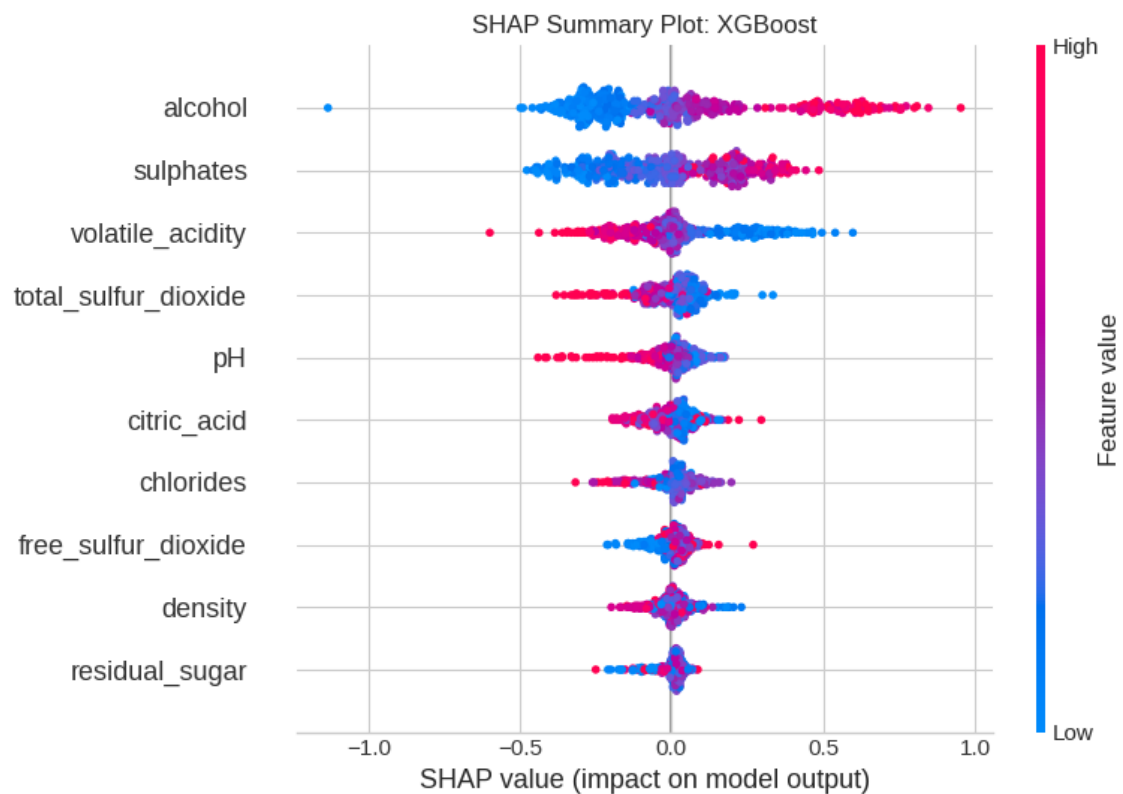


図 12 Summary Plot: XGBoost

図 12 より、アルコール度数(alcohol)が高いほど評価は高く、低いほど評価は低くなる
ことが分かる。この結果は決定木やランダムフォレスト(Random Forest)とも一致してい
る。また、ランダムフォレスト(Random Forest)のものよりも段階的に減っており、連続的
に見える。

XGBoost(eXtreme Gradient Boosting)モデルは、主要変数の影響方向を一貫して学習し
つつ、全変数を滑らかかつバランスよく統合する、最も安定した説明構造を持っているこ
とが分かる。

最後に SVM(Suport Vector Machine) について作成したグラフを以下の図 13 に Water
Fall Plot、図 14 に Bar Plot、図 15 に Summary Plot を示す。

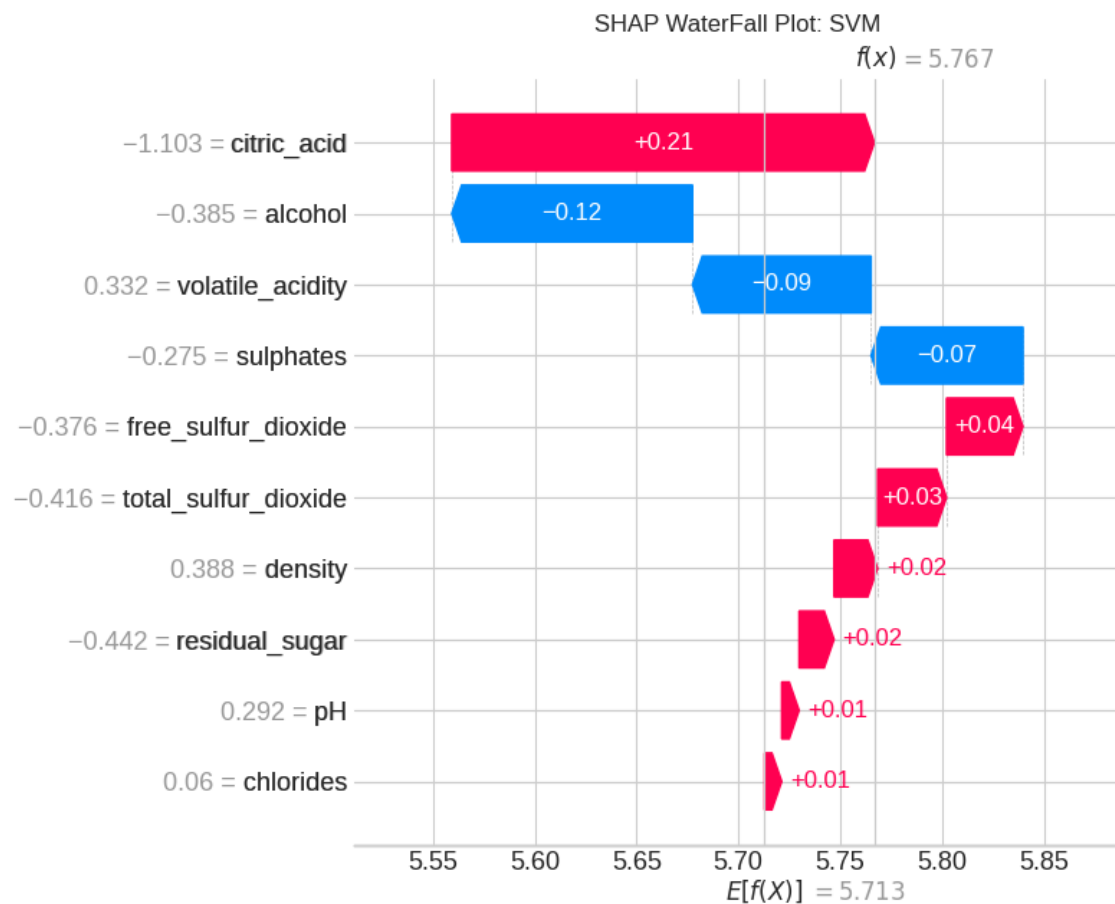


図 13 Water Fall Plot: SVM

図 13 より、このサンプルでは全体の予測値の平均より 0.05 程度大きな値が予測されている。このサンプルにおいてはクエン酸が最も大きな寄与を示しており、特に重要な 3 変数は全て負の寄与であった。SVM(Support Vector Machine)は他のモデルに比べてクエン酸(citric acid)への依存が大きいことが分かる。しかし、決定木と比べると様々な変数を利用していることが分かる。

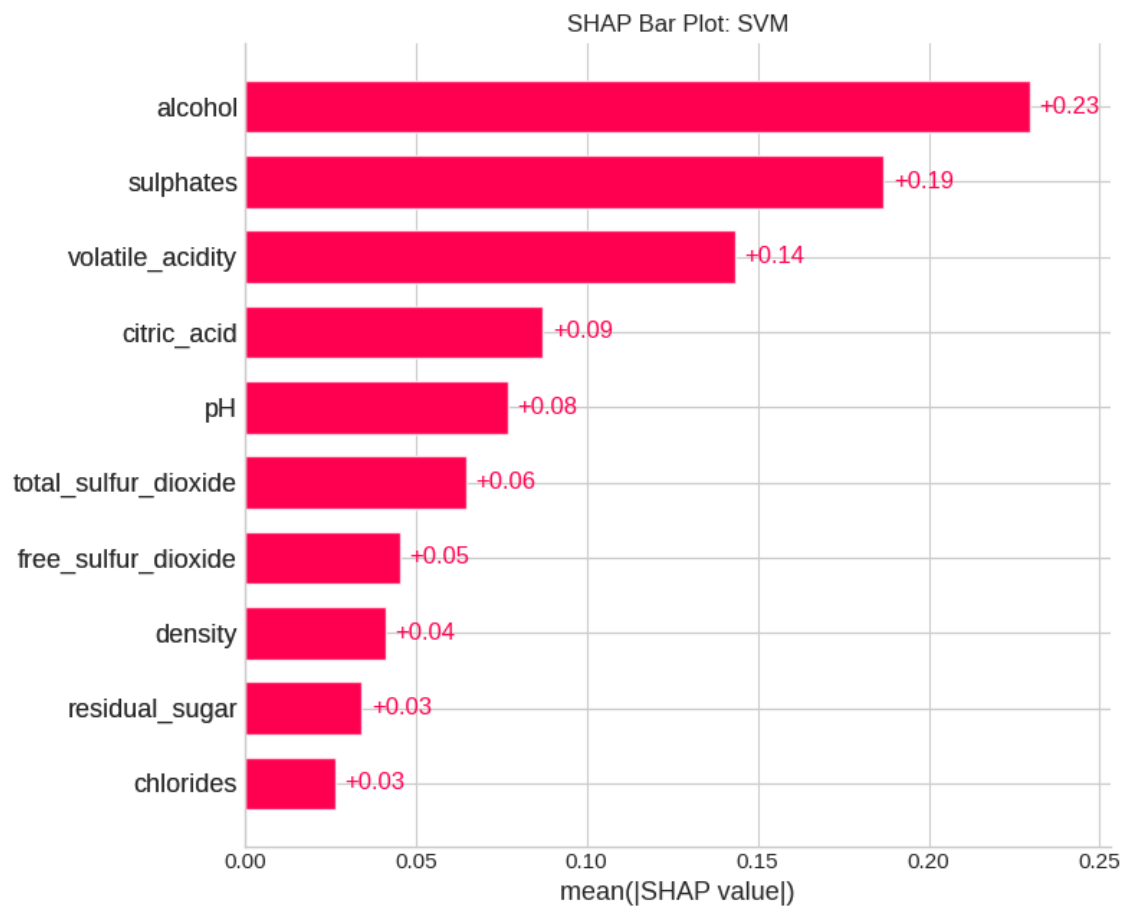


図 14 Bar Plot: SVM

図 14 より、SVM(Support Vector Machine)においても重要な変数の上位 3 変数は変わらなかった。また、XGBoost(eXtreme Gradient Boosting)と同様に様々な変数を均等に予測に使用していることが分かる。しかし、予測精度としては XGBoost(eXtreme Gradient Boosting)より劣っているものとなった。

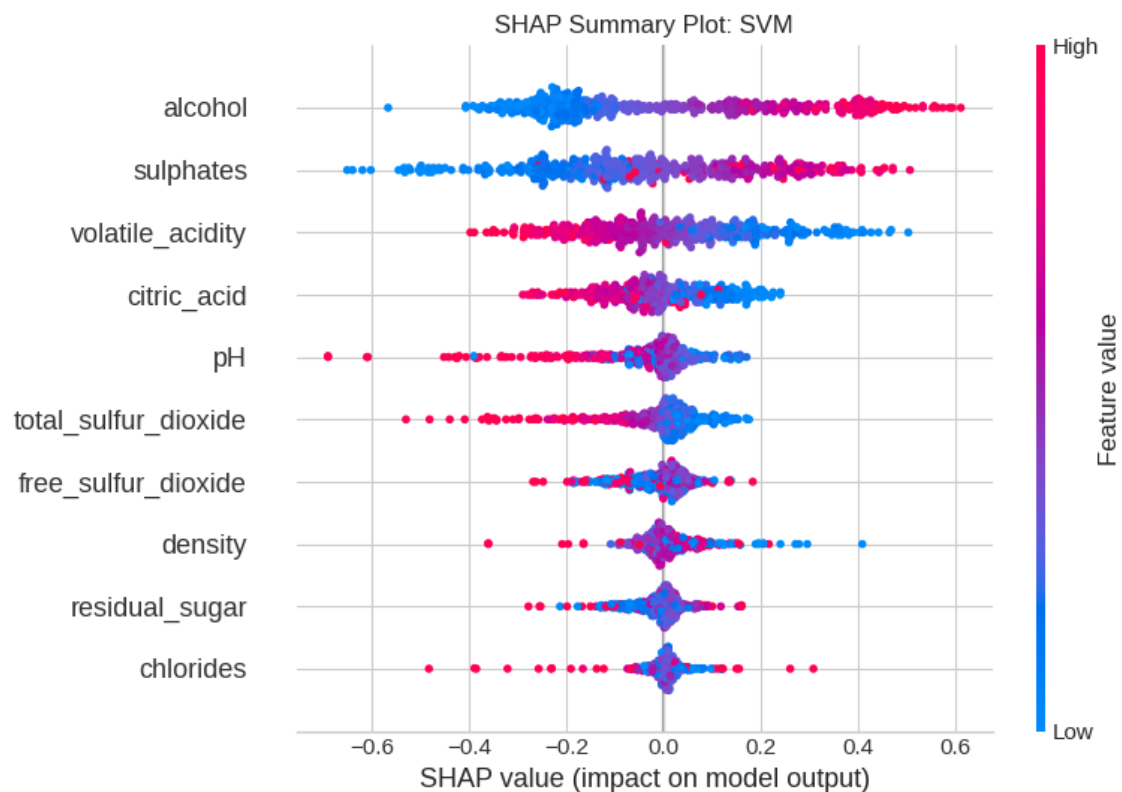


図 15 Summary Plot: SVM

図 15 より、アルコール度数(alcohol)が高いほど評価は高く、低いほど評価は低くなる
ことが分かる。この結果は決定木やランダムフォレスト(Random Forest)や
XGBoost(eXtreme Gradient Boosting)とも一致している。

SVM(Support Vector Machine)モデルは最も段階的に減っており、主要な変数だけでな
く、他の変数も統合的に使用しており、より安定的な説明モデルを持っていることが分か
る。

4.3. 考察

決定木、ランダムフォレスト(Random Forest)、XGBoost(eXtreme Gradient Boosting)、
SVM(Support Vector Machine)の 4 つの機械学習モデルを用いて変数重要度および、
SHAP 値の算出を行った。その結果としていくつかの共通した傾向と知見が得られた。

解析に用いたすべてのモデルにおいてアルコール度数(alcohol)が最も重要な変数として
特定された。また、揮発酸(volatile acidity)と硫酸塩(sulphates)がそれに次ぐ、重要変数と
して上位に来る傾向も一貫していた。モデルの構造に関わらず、これらの 3 変数が常に上
位に現れることが、これらがノイズや偶然ではなく、ワインの官能評価を決定付ける本質
的な化学的要因であることを示唆している。

SHAP 値の Summary Plot において、すべてのモデルでアルコール度数(alcohol)は強い正の相関を示した。アルコールはワインのコクや甘味の知覚に寄与しているからであると考ええる。また、揮発酸(volatile acidity)においてはすべてのモデルで負の相関がみられた。これは揮発酸が主に酢酸に由来し、過剰に含まれるとお酢のような不快臭の原因となるためであると考ええる。さらに、硫酸塩(sulphates)は、多くのモデルで正の相関がみられた。硫酸塩は酸化防止剤として添加されるものであるため、ワインの参加を防ぎ、ワインの鮮度を保てたものがよい評価になったと考える。

5. 結論

本レポートでは、赤ワインの理化学的特性データを用いて官能評価値を予測する機械学習モデルを構築し、その精度比較と要因解析を行った。

予測モデルの構築においてはランダムフォレスト(Random Forest)が最も高い予測精度を示した。これは、ワインの評価が複数の成分の複雑な相互作用によって決まるため、非線形な境界を柔軟に学習できるアンサンブル学習モデルが有効であったためである。一方、単体の決定木や SVM(Support Vector Machine)では十分な精度が得られず、モデル選択の重要性が確認された。

SHAP 値を用いた要因解析の結果、ワインの品質を決定づける最も重要な変数はアルコール度数(alcohol)であることがわかった。また、高品質なワインと評価されるには十分なアルコール度数を確保しつつ、不快臭の原因となる揮発酸を低く抑え、硫酸塩などで本質を適切に保持し続ける必要があることも分かった。

しかし、予測精度は最大でも 0.5 程度であった。これは今回使用した理化学的特性だけでは説明できない要因が官能評価に大きく影響していることを示唆している。特に評価者の体調や好みといった個人差や、香りや色味などの要因が影響しているからだと考える。

6. 使用したコード

```
7. import pandas as pd
8. import numpy as np
9. import matplotlib.pyplot as plt
10. import seaborn as sns
11.
12. from sklearn.model_selection import train_test_split,
    GridSearchCV
13. from sklearn.preprocessing import StandardScaler
14. from sklearn.linear_model import LinearRegression
```

```
15.     from sklearn.tree import DecisionTreeRegressor
16.     from sklearn.ensemble import RandomForestRegressor
17.     from sklearn.metrics import mean_squared_error, r2_score
18.     from sklearn.svm import SVC
19.     from sklearn.inspection import permutation_importance
20.
21.     import xgboost as xgb
22.
23.     import shap
24.
25.     from sklearn.datasets import fetch_openml
26.     plt.style.use('seaborn-v0_8-whitegrid')
27.     plt.rcParams['font.family'] = 'sans-serif'
28.     dataset = fetch_openml(name='wine-quality-red', version=1,
        as_frame=True, parser='auto')
29.     data = dataset.frame
30.
31.     df = data.rename(columns={'class': 'quality'})
32.     df['quality'] = df['quality'].astype(int)
33.     df.info()
34.     # 欠損値処理
35.     df.dropna()
36.
37.     # 外れ値のチェック
38.     # IQR 法
39.     name = 'residual_sugar'
40.     Q1 = df[name].quantile(0.25)
41.     Q3 = df[name].quantile(0.75)
42.     IQR = Q3 - Q1
43.     lower_bound = Q1 - 3 * IQR
44.     upper_bound = Q3 + 3 * IQR
45.
46.     outliers = df[(df[name] < lower_bound) | (df[name] >
        upper_bound)]
47.     print(f"{name}の外れ値")
48.     print(outliers[[name]])
```

```

49.     print(len(outliners))
50.
51.     # 箱ひげ図
52.     fig, (ax_box, ax_hist) = plt.subplots(2, 1, sharex=True,
53.                                           gridspec_kw={"height_ratios"
54.                                                         : (.15, .85)}),
55.                                           figsize=(10, 6))
56.     # 上段:箱ひげ図
57.     sns.boxplot(x=df[name], ax=ax_box, color='lightblue',
58.                 flierprops={"marker": "x", "markeredgecolor": "red"})
59.     ax_box.set(xlabel='') # 上の図の x 軸ラベルは消す
60.     # 下段:ヒストグラム
61.     sns.histplot(df[name], ax=ax_hist, kde=True,
62.                  color='skyblue')
63.     # タイトルなどの調整
64.     ax_box.set_title(f'Distribution & Outliers of "{name}"',
65.                      fontsize=14)
66.     plt.show()
67.     # 外れ値の削除
68.     df = df[(df[name] >= lower_bound) & (df[name] <=
69.        upper_bound)]
70.     target = 'quality'
71.     features = df.drop(columns=[target]).columns.tolist()
72.     # データの確認
73.     df.describe().T[['count', 'mean', 'std', 'min', '50%', 'max']]
74.     # 相関行列の確認(多重共線性のチェック)
75.     plt.figure(figsize=(10, 8))
76.     corr_matrix = df.corr()
77.     sns.heatmap(corr_matrix, annot=True, fmt=".2f",
78.                  cmap='coolwarm', square=True)
79.     plt.title("Correlation Matrix")

```

```

79.     plt.show()
80.     # 変数別にデータを分解
81.     df_del=df.drop(columns="fixed_acidity")
82.     features = df.drop(columns=[target,
    "fixed_acidity"]).columns.tolist()
83.
84.     y = df_del[target]
85.     X = df_del.drop(columns=[target])
86.     # データの分割
87.     X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.3, random_state=42)
88.     # 標準化
89.     scaler = StandardScaler()
90.     X_train_scaled = scaler.fit_transform(X_train)
91.     X_test_scaled = scaler.transform(X_test)
92.     # 結果の保存用辞書
93.     result = {}
94.     models = {}
95.     # モデルの構築
96.     # 決定木
97.     dt = DecisionTreeRegressor(max_depth=5, random_state=42)
98.     dt.fit(X_train, y_train)
99.     models['Decision Tree'] = dt
100.
101.    # ランダムフォレスト
102.    rf = RandomForestRegressor(n_estimators=100,
    random_state=42)
103.    rf.fit(X_train, y_train)
104.    models['Random Forest'] = rf
105.
106.    # XGBoost
107.    xgb_param_grid = {
108.        'n_estimators': [100, 200],
109.        'max_depth': [3, 5, 7],
110.        'learning_rate': [0.01, 0.1]
111.    }

```

```

112. xgb_reg = xgb.XGBRegressor(objective='reg:squarederror',
    random_state=42)
113. grid_search = GridSearchCV(xgb_reg, xgb_param_grid, cv=5,
    scoring='neg_mean_squared_error')
114. grid_search.fit(X_train, y_train)
115.
116. best_xgb = grid_search.best_estimator_
117. models['XGBoost'] = best_xgb
118. print(f"\nBest XGBoost Params: {grid_search.best_params_}")
119.
120. # SVM
121. svm = SVR(kernel='rbf')
122. svm.fit(X_train_scaled, y_train)
123. models['SVM'] = svm
124. # 予測と評価
125. metrics_data = []
126.
127. for name, model in models.items():
128.     if name == "SVM":
129.         y_pred = model.predict(X_test_scaled)
130.     else:
131.         y_pred = model.predict(X_test)
132.
133.     rmse = np.sqrt(mean_squared_error(y_test, y_pred))
134.     r2 = r2_score(y_test, y_pred)
135.
136.     metrics_data.append({'Model': name, 'RMSE': rmse, 'R2
    Score': r2})
137.
138. metrics_df = pd.DataFrame(metrics_data)
139. print(metrics_df)
140. # 結果の可視化
141. fig, axes = plt.subplots(1, 2, figsize=(14, 5))
142.
143. # RMSE 比較

```

```

144. sns.barplot(x='Model', y='RMSE', data=metrics_df,
               ax=axes[0], palette='viridis')
145. axes[0].set_title('Comparison of RMSE (Lower is Better)')
146. axes[0].set_ylim(0, metrics_df['RMSE'].max() * 1.2)
147.
148. # R2 比較
149. sns.barplot(x='Model', y='R2 Score', data=metrics_df,
               ax=axes[1], palette='viridis')
150. axes[1].set_title('Comparison of R2 Score (Higher is
               Better)')
151. axes[1].set_ylim(0, 1.0)
152.
153. plt.tight_layout()
154. plt.show()
155. # モデル間の変数重要度 比較分析
156. feature_importance_data = []
157.
158. # 決定木
159. dt_importance = models['Decision Tree'].feature_importances_
160. dt_imp = pd.Series(dt_importance,
                     index=features).sort_values(ascending=True)
161. for name, val in zip(features, dt_importance):
162.     feature_importance_data.append({'Model': 'Decision Tree',
                                     'Feature': name, 'Importance': val})
163.
164. rf_importance = models['Random Forest'].feature_importances_
165. rf_imp = pd.Series(rf_importance,
                     index=features).sort_values(ascending=True)
166. for name, val in zip(features, rf_importance):
167.     feature_importance_data.append({'Model': 'Random Forest',
                                     'Feature': name, 'Importance': val})
168.
169. xgb_importance = best_xgb.feature_importances_
170. xgb_imp = pd.Series(xgb_importance,
                     index=features).sort_values(ascending=True)
171. for name, val in zip(features, xgb_importance):

```



```

172.         feature_importance_data.append({'Model': 'XGBoost',
      'Feature': name, 'Importance': val})
173.
174.     # SVM
175.     svm_result = permutation_importance(svm, X_test_scaled,
      y_test, n_repeats=10, random_state=42)
176.     svm_importance = svm_result.importances_mean
177.     svm_imp = pd.Series(svm_importance,
      index=features).sort_values(ascending=True)
178.     for name, val in zip(features, svm_importance):
179.         feature_importance_data.append({'Model': 'SVM',
      'Feature': name, 'Importance': val})
180.
181.     df_importance = pd.DataFrame(feature_importance_data)
182.
183.     df_importance['Normalized Importance'] =
      df_importance.groupby('Model')['Importance'].transform(lambda x:
      x / x.max())
184.
185.     plt.figure(figsize=(14, 8))
186.     sns.barplot(x='Normalized Importance', y='Feature',
      hue='Model', data=df_importance, palette='viridis')
187.     plt.title('Feature Importance Comparison across Models
      (Normalized)')
188.     plt.xlabel('Relative Importance (Normalized to Max=1)')
189.     plt.show()
190.
191.     print(df_importance)
192.     # SVM (KernelExplainerを使用 - 計算コストが高いためデータの一部を使用)
193.     # 背景データとして訓練データの一部(kmeans で要約)を使用
194.     X_train_summary = shap.kmeans(X_train_scaled, 10)
195.
196.     for name, model in models.items():
197.         print(f"¥nAnalyzing: {name} ...")
198.
199.         plt.figure() # 新しい描画領域を作成

```

```
200.
201.     if name == "SVM":
202.         # explainer = shap.KernelExplainer(model.predict,
203.         X_train_scaled)
204.         explainer = shap.KernelExplainer(model.predict,
205.         X_train_summary)
206.         shap_values = explainer(X_test_scaled)
207.         shap_values.feature_names = features
208.     else:
209.         # 決定木ベースモデル用の Explainer (TreeExplainer)
210.         explainer = shap.TreeExplainer(model)
211.         shap_values = explainer(X_test, check_additivity=False)
212.         shap_values.feature_names = features
213.
214. plt.title(f"SHAP WaterFall Plot: {name}")
215. shap.plots.waterfall(shap_values[0], show=False)
216.
217. plt.tight_layout()
218. plt.show()
219.
220. plt.title(f"SHAP Bar Plot: {name}")
221. shap.plots.bar(shap_values, show=False)
222.
223. plt.tight_layout()
224. plt.show()
225.
226. plt.title(f"SHAP Summary Plot: {name}")
227. shap.summary_plot(shap_values, show=False)
228.
229. plt.tight_layout()
230. plt.show()
```